Progressive Retrieval Attention: Model-Bounded Sparse Native-KV for Long Context and URI-Addressed Memory

Jorge Simao
EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 10, 2026

Abstract

Progressive Retrieval Attention (PRA) gives a decoder sparse access to long logical context stored as URI-addressed memory. Compact gists index historical layer-native keys and values (K/V), and only selected full token K/V joins recent context under the decoder's existing attention softmax and output projection. Model-bounded encoding divides long sources into safe causal blocks before exposing smaller routing chunks. This distinction repairs an early fragmentation failure: contextual/native-KV slicing recovers 0.998–1.000 of dense context benefit through 256 addressable units while activating 4.4–5.1% of accessible K/V at fixed selection breadth. A packed exact router reduces warm 256-way selection to 5.5–11.6 ms. Under a hard 32-token native-operation limit, a five-seed probe addresses a 184-token displaced prompt head; at 192 logical tokens, routed loss is 1.0567 versus 1.2287 for truncation and 1.7089 for independent block encoding. These controlled results establish sparse native-KV transport, exact routing, and model-bounded long-context execution. They do not establish unrestricted question answering or end-to-end production efficiency.

1 Introduction

Progressive Retrieval Attention (PRA) is motivated by a common mismatch in long-context language modeling. Many tasks require access to large context, but the relevant evidence is sparse, hierarchical, and often discovered only after partial reasoning. A conventional decoder-only transformer receives a flat token sequence. PRA instead treats references as an index over model-native historical K/V and activates only selected positions. References may denote prior tokens, external objects, or an implicit prompt head; retrieval-augmented generation is therefore one possible source of memory, not the definition of PRA [10].

Here, *URI-addressed memory* means external source state identified by a stable URI-like reference, represented compactly for routing, and selectively materialized as layer-native K/V. Explicit URI references name external objects. The reserved `#__head` and streaming history are implicit addressable-memory forms that reuse the same chunk, gist, routing, and materialization machinery without adding visible reference tokens.

Suppose a 32K-token diagnostic record contains a fact needed near the end, but the base model can directly process only 4K tokens. PRA keeps the recent 4K tokens local, registers the displaced 28K as a request-local object named `#__head`, encodes that object in model-safe blocks, and exposes smaller chunks through a routing index. The query ranks gists, but attention reads the selected chunks' original token-level K/V. The base model is never asked to process the entire 32K logical context in one operation.

This paper documents a standalone PyTorch prototype designed as a controlled experimental instrument, not a production long-context model. Its main contributions are:

- a weight-preserving native-K/V transport in which selected memory and direct context share one attention normalization;
- a diagnosis of high-fragmentation failure that separates contextual *encoding blocks* from smaller addressable *routing chunks*;
- an exact packed router, persistent routing indexes, whole-chunk materialization budgets, row-private batching, and optional CPU-resident source K/V;
- a model-bounded implicit-head path that enforces a hard per-operation context limit and rolls generated history into routable memory; and
- five-seed controlled probes and counterfactual metrics that distinguish transport, routing, content causality, active attention K/V, resident K/V, and transfer cost.

PRA in 60 Seconds

Consider the exact bounded-context setting used later in the paper. A 192-token logical prompt is six times a hard 32-token native limit. PRA keeps the last eight tokens direct and registers the preceding 184 tokens—5.75 times the native limit—as the request-local `#_head`. It encodes that head in 16-token causal cores with up to four tokens of left context, so the largest observed encoding call is 20 tokens, or 62.5% of the hard ceiling. Context-only overlap is discarded, and each core's native K/V is sliced into four-token routing chunks. One mean gist per chunk enters the packed index. Exact routing ranks eight candidates, and a 20-token whole-chunk budget admits at most five of them. The resulting allocation obeys $8_{\text{direct}} + 20_{\text{memory}} + 4_{\text{reserve}} = 32_{\max}$.

In one line, the data path is:

$$\text{logical } 192 \to \text{bounded encoding calls} \le 20 \to \text{4-token routing chunks} \to \text{mean gists/index} \to \text{exact top-}k \to \text{budgeted native K/V} \to \text{direct + memory attention.}$$

The 32-token ceiling constrains every base-model operation; it does not imply that all 192 tokens are attended to simultaneously.

The implementation consists of four main components. First, a tiny GPT-style language model provides causal decoder blocks. Second, a reference system maps handles through an explicit local table to URI-addressed objects; the prompt adapter can also create a reserved implicit reference without adding visible tokens. Third, a resolver encodes each object in bounded blocks and stores full per-layer token K/V in smaller routing chunks, together with one or more gists. Fourth, a PRA attention layer performs hierarchical reference-then-chunk routing and inserts selected native token K/V into ordinary causal attention. An optional scaled cross-attention branch preserves earlier experiments.

The easiest way to follow the implementation is as a prepare-then-use pipeline. During preparation, each batch row resolves its own handles, encodes the resulting chunks with the same Transformer stack used for prompts, and records two representations at every PRA layer: a small gist for routing and full token K/V for reading. During use, all prompt rows pass through the Transformer together. The last query in each row ranks only that row's gists; selected chunks are materialized as variable-length memory; and their original token K/V is prepended to local K/V under one softmax. Local causal attention never disappears. The optional legacy mode instead reads the same memory through a separately normalized cross-attention residual.

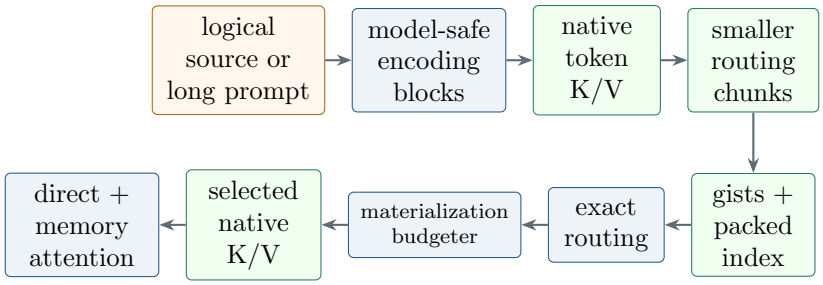

Figure 1: End-to-end PRA pipeline. Large logical sources are contextualized in bounded encoding blocks, then sliced into smaller addressable routing chunks. Gists select chunks; the budgeter admits complete native token K/V; and selected memory joins direct context in ordinary attention.

PRA in six distinctions. PRA is not RAG preprocessing. A reference names available context; it does not materialize it. A gist is a routing key, not the memory read by attention. Top-k selects chunks, not individual tokens. A selected chunk materializes full native token K/V by default. Logical context may exceed native model context, but no base-model operation may exceed the configured native limit.

Figure 1 gives the complete data path. The remainder of the paper follows the discovery sequence: exact transport and first probes; the unexpected fragmentation failure; its correction through contextual slicing; scaling and routing optimization; then residency, bounded logical context, and streaming rollover. Historical cross-attention results are retained later as a separate transport comparison.

2 Model Definition

2.1 Decoder Backbone

Let $x_{1:T}$ be a token sequence. The model embeds tokens and absolute positions:

$$H^{(0)} = E_{\text{tok}}(x_{1:T}) + E_{\text{pos}}(1:T). \tag{1}$$

For each layer $\ell \in \{0, \ldots, L-1\}$, the implemented block is a pre-normalized residual decoder block:

$$\tilde{H}^{(\ell)} = \text{LN}_1(H^{(\ell)}), \tag{2}$$
$$A^{(\ell)} = \text{PRAttn}_\ell(\tilde{H}^{(\ell)}), \tag{3}$$
$$U^{(\ell)} = H^{(\ell)} + A^{(\ell)}, \tag{4}$$
$$H^{(\ell+1)} = U^{(\ell)} + \text{FFN}_\ell(\text{LN}_2(U^{(\ell)})). \tag{5}$$

The feed-forward network is a two-layer MLP with GELU activation and configurable hidden width d_{ff}, defaulting to $4d$. The output head maps the final normalized hidden state to vocabulary logits. Configurable d_{ff} permits close parameter matching without changing attention width or depth.

2.2 Causal Self-Attention

For hidden states $X \in \mathbb{R}^{B\times T\times d}$, the local branch computes multi-head causal self-attention:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \tag{6}$$

$$O_{\text{local}} = W_O \,\text{merge}\left[\text{softmax}\left(\frac{QK^\top + M_{\text{causal}}}{\sqrt{d_h}}\right)V\right]. \tag{7}$$

The implementation stores a lower-triangular causal mask with maximum sequence length configured by `PRAConfig.max_seq_len`.

2.3 Minimal PRA Implementation

A minimal reproduction needs no recursive graph, multi-gist pooler, CPU offload, or streaming policy. It needs one historical or external source, fixed routing chunks, one mean key gist per chunk and layer, exact normalized-dot-product top-k, complete native K/V for selected chunks, and one shared attention softmax. The source is encoded causally; for an exact displaced-prefix comparison, its positions are historical and the direct tail continues at the displaced-source offset. The following single-row pseudocode keeps layer specificity and tensor shapes explicit:

```
01 | # Preparation: one causal source pass, captured separately at every layer l.
02 | Ksrc[l], Vsrc[l] = capture_native_kv(source_ids, layer=l)  # [1,H,S,Dh]
03 | for c, (a, b) in enumerate(fixed_slices(S, route_tokens)):
04 |     Kc[l,c], Vc[l,c] = Ksrc[l][:,:,a:b,:], Vsrc[l][:,:,a:b,:]
05 |     G[l,c] = normalize(merge_heads(Kc[l,c]).mean(dim=1))[0] # [D]
06 |
07 | # Use: q/k/v are the layer's ordinary projections of the direct sequence.
08 | q, kd, vd = project_qkv(x, layer=l)                        # [1,H,T,Dh]
09 | qr = normalize(merge_heads(q[:,:,-1:,:])).squeeze(1)       # [1,D]
10 | score = qr @ stack(G[l]).T                                 # [1,C]
11 | chosen = torch.topk(score, k=min(k, C), dim=-1).indices
12 | km = cat([Kc[l,c] for c in chosen[0]], dim=2)              # [1,H,M,Dh]
13 | vm = cat([Vc[l,c] for c in chosen[0]], dim=2)              # [1,H,M,Dh]
14 | keys, values = cat([km, kd], 2), cat([vm, vd], 2)
15 | visible = cat([ones(T,M), tril(ones(T,T))], dim=-1)
16 | logits = (q @ keys.transpose(-2,-1)) / sqrt(Dh)            # [1,H,T,M+T]
17 | heads = softmax(mask(logits, visible), dim=-1) @ values
18 | y = W_o(merge_heads(heads))                                # existing W_o
```

Lines 2–5 build a layer-specific index whose gists are selectors, not transported memory. Lines 10–11 compute cosine scores as normalized dot products and use exact `topk`; no approximate index is involved. Lines 12–18 materialize complete token-level K/V for selected chunks. Historical memory precedes the direct sequence and is visible to every direct query, while the direct portion retains its causal triangle. Both sources share one normalization and the existing W_O.

The full prototype grows from this minimal core in an explicit sequence:

minimal PRA → URI-addressed multiple references → encoding/routing separation → packed exact routing → bounded `#__head` → materialization budget → CPU residency → streaming rollover.

Each extension changes source management or systems policy; none changes the canonical contract that selected layer-native K/V and direct K/V meet under one softmax.

3 Reference Graph and Runtime State

3.1 Reference Graph

Let the external context be a directed attributed graph

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}), \tag{8}$$

where each node $v \in \mathcal{V}$ is a tuple (u, z, s, m, R) containing a stable URI u, content z, optional summary s, metadata m, and an explicit local reference table R. An edge $(v_i, a, v_j) \in \mathcal{E}$ exists only when $R_i[a] = u_j$; URI-prefix scans do not silently invent children. This local namespace makes recursive dependencies inspectable and prevents unrelated objects with similar URI prefixes from entering the graph.

For input sequence x, the reference table exposes a candidate set $\mathcal{H}(x) \subseteq \mathcal{V}$. A resolver

$$\rho(u, p, t) \to (z, s, R, m, \nu) \tag{9}$$

maps URI u, authorization policy p, and logical time or version t to a resolved object. Including p and t in the interface clarifies two systems requirements that the in-memory prototype currently simplifies: access control and freshness.

3.2 Selection and Expansion Operators

At layer ℓ, document u is partitioned into chunks $c \in \mathcal{C}_u$. Encoding produces full token memory $(K_{u,c,\ell}, V_{u,c,\ell})$ and paired gist sets $G^K_{u,c,\ell}, G^V_{u,c,\ell} \in \mathbb{R}^{G_c \times d}$. The default mean mode has $G_c = 1$: it reconstructs the projected heads into d dimensions and pools over chunk tokens,

$$g^K_{u,c,\ell} = \frac{1}{|c|} \sum_{j \in c} \text{mergeheads}(K_{u,c,\ell,j}). \tag{10}$$

Last-token, explicit reference-end, and a registered GRU pooler also produce one gist. Deterministic k-means, diversity-selected prototypes, a line-topology self-organizing map, and a global/local hybrid may produce several. Multi-gist modes derive value gists from the same assignments as their key gists, preserving the regions that each routing key names. The routing query is the last valid projected query reconstructed across heads,

$$q_{t,\ell} = \text{mergeheads}\left(X^{(\ell)} W_Q^{(\ell)}\right)_t. \tag{11}$$

For $g^K_{u,c,\ell,j} \in G^K_{u,c,\ell}$, the router computes

$$a_{u,c,\ell,j} = \cos(q_{t,\ell}, g^K_{u,c,\ell,j}), \qquad a_{u,c,\ell} = \text{Agg}_{j=1}^{G_c} a_{u,c,\ell,j}, \tag{12}$$

with maximum aggregation by default and mean or log-sum-exp alternatives. Reference-level gist sets are constructed separately from chunk gist sets, allowing a true URI-first stage before chunk scoring. Hierarchical search selects at most k_r references and then at most k_c chunks independently inside each retained reference. A threshold is applied without converting the two budgets into one flattened top-k. A global-chunk strategy is available for ablation but still enforces both constraints. An expansion operator

$$\mathcal{X}(A, \mathcal{G}, b, d) \to A' \tag{13}$$

maps active set A to an expanded set A' under branching budget b and depth budget d. The corrected cache builder implements bounded child-first expansion. It follows only each resolved object's local table, shares reference and token budgets across the traversal, records build states and dependencies, and applies explicit cycle, missing-reference, depth, and overflow policies. This deterministic expansion is a runtime policy, not yet a learned iterative controller.

3.3 Cache and Lifecycle State

The runtime state before token t is

$$\Omega_t = (\mathcal{G}, \mathcal{H}_t, A_t, \mathcal{C}_t, p_t, \nu_t), \tag{14}$$

where A_t is the active reference set, $\mathcal{C}_t$ the resident encoded cache, p_t policy state, and ν_t provenance/version state. A cache entry is keyed conceptually by $(u, v, \ell, m, \theta, p)$: URI, content version, layer, model identity, relevant encoder parameters, and policy domain. The prototype key is only URI because each cache is rebuilt per example; persistent or shared caches require the fuller identity.

One runtime transition can be written

$$A_t = \mathcal{S}_{k,\tau}(q_t, \mathcal{C}_t), \tag{15}$$
$$A'_t = \mathcal{X}(A_t, \mathcal{G}, b, d), \tag{16}$$
$$\mathcal{C}'_t = \mathrm{admit}(\mathrm{encode}(\rho(A'_t))), \tag{17}$$
$$(y_t, \Omega_{t+1}) = \mathrm{attend}(x_{\le t}, \mathcal{C}'_t, \Omega_t). \tag{18}$$

In the corrected prototype, resolution and encoding happen before the batched prompt forward. Each row's cache is ephemeral and private, and a batch-aware wrapper preserves those namespaces while the prompt itself executes once as a tensor of shape $[B, T]$. Recursive construction is available, but admission retains every successfully built entry within fixed budgets and expansion is not learned. Entries move through explicit `unseen`, `building`, `ready`, and failure states so partially built cyclic entries never become searchable. Persistent admission, eviction, and cross-request reuse remain outside the implementation.

4 Reference Handles and Resolver

4.1 Reference Tokens

The prototype uses explicit textual handles of the form `<REF_n>`. A regular expression parser extracts occurrences and resolves them only through the current object's `ReferenceTable`. Each table entry stores an id, token, URI, optional summary, and metadata. Legacy `!!ref:uri!!` syntax is retained only for compatibility.

4.2 URI and Anchor Semantics

References point to simple URIs such as `mem://doc1`. Anchored locations may use URI fragments, for example `mem://doc1#section.a`, but lexical URI ancestry is not semantic graph ancestry. The in-memory resolver stores structured document records containing text, an optional summary, metadata, version, anchors, and a local reference table. A missing URI raises an explicit error; missing-reference policy is owned by the recursive builder rather than hidden inside resolution.

4.3 Resolved References

Given a handle r with URI u, the resolver returns

$$\text{resolve}(u) = (\text{text}_u, \text{summary}_u, R_u, \text{metadata}_u, \text{version}_u). \tag{19}$$

This simple resolver is sufficient for synthetic and local experiments. Future systems can replace it with file, database, vector-index, search, or web resolvers while preserving the same high-level interface.

4.4 Implicit Prompt-Head Reference

The same resolver/cache contract can retain prompt history that no longer fits the local window. Let N be the exact input-token count and let

$$D = \min(\texttt{max\_prompt\_direct\_tokens}, L_{\text{model,max}} - L_{\text{reserve}}), \tag{20}$$

where `model_max_context_tokens` defines the hard native limit $L_{\text{model,max}}$ and an unset direct limit inherits the remaining native capacity. In `implicit_reference` mode, an input with $N > D$ is split into head IDs $x_{1:N-D}$ and direct-tail IDs $x_{N-D+1:N}$. The adapter registers the head under the reserved URI `pra://implicit/prompt/head` and diagnostic handle `#__head`, then passes its token IDs directly to ordinary chunking, cache construction, gist routing, and selected-memory attention. It does not decode and retokenize the head or insert a synthetic marker into the visible tail.

Each batch row owns its implicit object even though all rows reuse the same URI string. Mixed tail lengths are padded with a validity mask, and the routing query is taken from the last valid position rather than the final rectangular column. Explicit references remain in the same row-private cache. The prompt head may use a separate chunk cap via `max_prompt_gists`; `None` retains every prompt-head chunk, while chunk length itself remains bounded by the model context limit. The alternative overflow policies preserve the prior behavior: `truncate` keeps only the recent tail and `error` rejects the request.

Preparation reports the total, direct, implicit, chunk, and gist counts together with whether overflow occurred. The reserved object is invalidated between requests, and an explicit reference cannot claim its URI. During generation, a completed routing-sized prefix of the direct tail is moved into the same request-local head whenever appending a token would exceed D. The head entry and its packed routing indexes are rebuilt after invalidation; this correctness-first implementation is not yet an incremental append cache. Direct history therefore stays bounded while generated history remains logically represented and routable.

5 Layer-Specific Memory Cache

5.1 Cache Entries

The PRA cache stores a reference entry containing layer-specific chunk memory:

$$C_u = \left(u, z_u, m_u, \left\{\left(c, K_{u,c,\ell}, V_{u,c,\ell}, G^K_{u,c,\ell}, G^V_{u,c,\ell}\right)\right\}_{c,\ell}\right). \tag{21}$$

Each chunk records a deterministic identifier, source URI, token interval, metadata, full projected token K/V, and a layer-specific paired gist set. K/V and gists are detached by default; a trainable-gist mode preserves the registered GRU gist path. Optional summary evidence, when present, is projected contextually into the same layer space and attached to the chunk

gist set. The implementation exposes an abstract `PRAMemoryCache` and a dictionary-backed `PRASimpleMemoryCache`.

5.2 Reference Encoding Pass

Before a batch forward pass, the training code builds isolated caches from batch metadata. For each referenced document:

1. Resolve the URI to text, optional summary, metadata, and its explicit local reference table.
2. Recursively ensure children are ready first when recursive references are enabled, subject to shared depth, reference, and token budgets.
3. Partition the text with `none`, bounded fixed windows, explicit markers, or a supplied semantic partitioner. Semantic mode without a plugin fails explicitly.
4. Run each document through the same model path. Parent encoding may read already-ready child memory; nonrecursive encoding disables PRA memory.
5. At every PRA layer, project each chunk's token states through that layer's W_K and W_V, then derive paired key/value gist sets under the configured strategy.
6. Store provenance, fingerprints, truncation details, layer-specific chunk tensors, and dependency events before atomically marking the entry ready.

The fourth and fifth steps are the important representation boundary. Reference text is not mapped directly to a single static document vector. A chunk starts at the embedding layer, advances through the model, and exposes the K/V that the corresponding PRA layer would consume. Consequently a lower-layer cache and a higher-layer cache describe the same source at different model depths. One or more routing gists are then pooled from the projected keys at that depth. They are suitable for inexpensive ranking; they do not replace the detailed token memory.

Fixed windows can overlap, but selected materialization deduplicates overlapping source-token spans. Maximum gist counts and overflow policies bound routing state. Cache fingerprints include content, model, tokenizer, and routing identities so a persistent implementation can invalidate stale representations. The current per-example cache remains ephemeral, but the identity rules are explicit.

The implementation also separates *routing units* from *encoding context*. In `block_slice` mode, consecutive URI regions are contextualized together and their native K/V is sliced back into independently addressable entries. In `native_slice` mode, the complete historical source is encoded once before slicing. Source positions may be local to each block or preserved globally. For exact historical semantics with learned absolute positions, the direct tail must continue after the displaced source rather than restart at zero; `prompt_position_mode=historical` implements this continuation. A model-level test confirms that full historical K/V plus the continued tail reproduces dense SelfAttention tail logits to numerical tolerance.

5.3 Four Context Scales and Bounded Encoding

Whole-history `native_slice` is exact only while that history fits the underlying model. The bounded path makes four lengths explicit:

$$L_{\text{logical}} \gg L_{\text{model,max}} \geq L_{\text{encode}} > L_{\text{route}}. \tag{22}$$

The same `ChunkingConfig` abstraction configures both levels, but with independent instances. `encoding_chunking` partitions a long source into model-safe causal cores; `routing_chunking` slices each core's captured native K/V into smaller addressable units. Consequently one base-model call can produce several routing chunks. Fixed, marker, and supplied semantic partitioners share one implementation rather than maintaining separate reference and prompt splitters.

Here L_{logical} is everything the request can name, $L_{\text{model,max}}$ is the largest sequence accepted by any one base-model operation, L_{encode} is one contextual encoding block, and L_{route} is one addressable K/V slice. For example, the bounded experiment uses a 192-token logical prompt, a 32-token model limit, 16-token encoding cores with up to four context tokens, and four-token routing chunks. **Encoding granularity and routing granularity are distinct: a routing chunk is an addressable retrieval unit, not necessarily an appropriate independent encoding unit.**

Encoding overlap is context-only. For core interval $[a, b)$ and left context h, the model receives $x_{a-h:b}$ but stores only the K/V slice corresponding to $x_{a:b}$. Logical source offsets remain global in cache metadata; learned absolute position offsets are used only when the complete requested range fits the native table, otherwise the bounded block uses local positions. Every encoding call validates $|x_{a-h:b}| \leq L_{\text{model,max}}$. This preserves useful boundary context without duplicating overlap in stored memory, but it approximates whole-history causal encoding once $L_{\text{logical}} > L_{\text{model,max}}$.

6 PRA Attention

6.1 Hierarchical Content Routing

At generation or training time, each batch row independently compares its projected last-valid-token query with layer-specific chunk gists. For query $q_{b,t,\ell}$ and chunk gist set $G^K_{u,c,\ell}$, the default content score is

$$\text{score}(u, c \mid q) = \max_{g \in G^K_{u,c,\ell}} \frac{q^\top g}{\|q\|_2 \|g\|_2}. \tag{23}$$

This score is a coarse admission decision: it asks which chunks are worth opening for the current row and layer. It is distinct from the token-level attention weights computed after a chunk is opened. The default hierarchical policy uses independent reference and per-reference chunk top-k settings. They are configured as `top_k_references` and `top_k_chunks_per_reference`; either may be zero, yielding no memory. `top_k_refs` remains a warning-producing compatibility alias for one release. Summaries are disabled by default. If enabled, `replace`, `hybrid`, and `augment` policies combine projected summary evidence with content scores; a missing summary always falls back to content routing.

6.2 Canonical Native-KV Transport

For selected chunk set $\mathcal{A}_{b,\ell}$ and layer ℓ, the implementation materializes only selected full-token memories for each batch row:

$$K^{\text{mem}}_{b,\ell} = \text{concat}_{(u,c)\in\mathcal{A}_{b,\ell}} K_{u,c,\ell}, \quad V^{\text{mem}}_{b,\ell} = \text{concat}_{(u,c)\in\mathcal{A}_{b,\ell}} V_{u,c,\ell}. \tag{24}$$

With batch size B, H heads, head width D_h, direct length T, and selected memory length M_b, the query and local K/V have shape $[B, H, T, D_h]$. Each cached chunk stores $[1, H, M_c, D_h]$ K/V; concatenation produces row-specific memory $[1, H, M_b, D_h]$ and attention logits $[1, H, T, M_b + T]$.

The routing query has shape $[B, D]$, the packed gist index $[C, G, D]$, chunk scores $[B, C]$, and selected chunk indices $[B, k]$, where $D = HD_h$. These routing tensors decide what to open; they do not replace the token dimension M_b consumed by attention.

It then prepends memory K/V to local K/V and computes one causal normalization:

$$O = W_O \,\mathrm{merge}\left[\mathrm{softmax}\left(\frac{Q_b[K^{\mathrm{mem}}_{b,\ell}; K^{\mathrm{local}}_{b,\ell}]^\top}{\sqrt{d_h}} + M_{\mathrm{causal}}\right)[V^{\mathrm{mem}}_{b,\ell}; V^{\mathrm{local}}_{b,\ell}]\right]. \tag{25}$$

Memory positions are visible to every local query; local positions retain their ordinary causal and padding masks. There is no $W_{O,\mathrm{mem}}$, separate softmax, or memory scale. If memory is disabled or selection is empty, the implementation takes the exact local path. Routing gists index token K/V and are not themselves transported, except in the explicit `gist_only` diagnostic.

Routing gists are selectors, not transported memory; selected chunks materialize full native token-level K/V. Four resource quantities must therefore remain separate: the gist/index footprint, all stored source K/V, K/V transferred for a request, and K/V active in that request's attention. A lower active fraction is not automatically a lower total cache footprint.

The normal `selected_chunks` mode transports exactly the token K/V belonging to the routed chunks. Two diagnostic modes make the boundary testable: `full_reference` opens every chunk belonging to a selected URI, while `gist_only` supplies one K/V position per selected chunk. The first removes chunk selection from the experiment; the second asks how much can be done without token-level detail. Neither changes the URI or reference-ranking stage.

6.3 Global Whole-Chunk Materialization Budget

Routing top-k produces candidates, not a promise that every candidate can enter native attention. For a row with T physical direct tokens, the allocator computes

$$B_{\mathrm{mem}} = \min\left(B_{\mathrm{configured}}, L_{\mathrm{model,max}} - T - L_{\mathrm{reserve}}\right) \tag{26}$$

and rejects a negative remainder. Candidates from `#__head`, explicit references, and recursive memory share this one budget. They are sorted by decreasing chunk score with URI and chunk-identifier tie-breaks. A whole chunk is admitted if its token cost fits the remaining budget; an oversized chunk is skipped so a smaller lower-ranked chunk may still fit. Budgeting precedes host-to-device transfer, ensuring a rejected CPU-resident chunk is never moved. Every native attention call therefore obeys

$$T + M_{\mathrm{materialized}} + L_{\mathrm{reserve}} \le L_{\mathrm{model,max}}. \tag{27}$$

Partial chunks, compression, and summary substitution intentionally remain separate materialization policies.

Variable memory lengths are executed in one of three modes. Mode zero evaluates each nonempty row independently; mode one pads a single masked rectangle; and mode $K \ge 2$ deterministically partitions rows into at most K contiguous length buckets using either equal-count grouping or a dynamic-programming minimum-padding plan. Masks preserve numerical equivalence, empty rows remain zero, and outputs are restored to original batch order. Diagnostics report actual bucket count, selected versus padded positions, and allocation ratio.

6.4 Optional Adapted Cross-Attention Transport

Setting `memory_transport=cross_attention` restores the earlier separately normalized branch, output projection, and residual scale. Legacy checkpoint booleans migrate to this mode, so his-

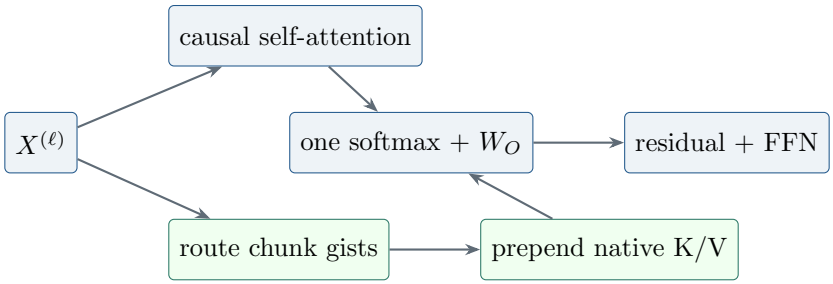

Figure 2: Canonical PRA block. Selected native K/V and local K/V share one normalization and the existing output projection.

torical reports remain reproducible. This mode changes the model and requires adaptation; it is retained as a later transport comparison rather than the definition or default of PRA.

7 Complexity and Resource Model

Let L be the number of decoder layers, L_p the number of PRA-enabled layers, T local sequence length, r the number of resolved references, m_i the token length of reference i, $M = \sum_{i \in A} m_i$ the selected resident memory length, d hidden width, and w bytes per cached scalar. Ignoring heads and constant factors, local decoder attention costs

$$O(LT^2d). \tag{28}$$

The prototype separately encodes every resolved reference through the model, costing approximately

$$O\left(Ld\sum_{i=1}^{r} m_i^2\right) + O\left(L_p d^2 \sum_{i=1}^{r} m_i\right) \tag{29}$$

for independent reference self-attention and layer-specific K/V projection. Native memory attention adds

$$O(L_p TMd) \tag{30}$$

to a full-sequence forward pass. The resident K/V payload is approximately

$$B_{\text{cache}} = 2wL_pMd \tag{31}$$

bytes, excluding summaries, object metadata, allocator overhead, and duplicated entries. Canonical native transport adds no trainable projection. The optional cross-attention mode adds one $d \times d$ memory output projection plus bias per PRA layer.

For an overlong prompt with displaced head length $P = N - D$, let nonoverlapping encoding cores have lengths $e_j \le L_{\text{encode}}$ and context-only overlaps h_j. The direct local term becomes $O(LD^2d)$, while cold construction adds $O(Ld\sum_j (e_j + h_j)^2)$ plus K/V projections. Routing scores $G = \sum_c G_c$ small vectors, and selected-memory attention still depends on transported token count M. This decomposition is a measurement plan, not an efficiency claim: the current streaming implementation rebuilds the bounded head after rollover and can therefore cost more than an incremental native cache or strong long-context baseline.

These costs explain why shorter visible prompts alone are not a sufficient efficiency result. If every reference is encoded once per example and never reused, $\sum_i m_i^2$ can dominate. PRA becomes attractive only when selection keeps M small, encoding is amortized across tokens or requests, summaries avoid unnecessary resolution, or quality improves under a fixed compute budget. A systems evaluation must decompose resolver time, tokenization, reference encoding, selection, memory attention, local model compute, cache transfer, and synchronization.

The current batch-aware cache wrapper preserves semantic isolation and permits one $[B, T]$ prompt forward. Reference construction and row-local routing still iterate over logical rows, so they are not yet fully vectorized. A production design would also pack reference encodings and routing candidates with row offsets or masks. Its complexity can approach the same aggregate arithmetic while reducing launch and orchestration cost, provided selection and cache identity remain isolated.

8 Dataset and DataLoader Pipeline

8.1 Dataset Schema

Datasets are represented by three JSONL files per stage:

- `documents.jsonl`: URI-addressed documents, optional summaries, titles, anchors, metadata, and local reference tables.
- `references.jsonl`: reference ids, URIs, summaries, anchors, and metadata.
- `questions.jsonl`: prompts, answers, expected reference ids/URIs, expected chunk ids or spans, and expected anchors.

The loaded sample type is `QuestionSample`, containing a question, answer, list of `ReferenceSample` objects, optional reference/chunk relevance labels, and metadata. Chunk labels are optional: chunk metrics are omitted rather than fabricated when labels are absent. New dataset generators do not emit summaries; loaders retain backward compatibility with legacy optional summary fields and explicit summary experiments. The current registry includes synthetic memory QA, hierarchical synthetic references, code repositories, Wikipedia, books, technical documentation, and GitHub-style repositories.

8.2 Collation

The `PRACollator` tokenizes each question and answer, forms next-token labels, pads variable-length rows, builds a per-sample reference table, and preserves non-tensor metadata. Prompt labels are replaced by the padding/ignore id, so reference-conditioned loss is computed only over answer-continuation tokens. This metadata is essential: the training step uses it to resolve and encode references before the main model forward pass.

8.3 Data Module

The `PRADataModule` used by synthetic and repository-style stages builds a compact tokenizer from questions, answers, summaries, and reference texts. The WikiText-2 study instead trains a 2,000-token BPE tokenizer with whitespace pre-tokenization and reserves `<REF_1>` through `<REF_5>`. The serialized phase-1 tokenizer is restored for phase 2 and for post-fine-tuning plain-text evaluation. This prevents vocabulary drift from masquerading as an architectural effect.

8.4 Reference-Conditioned WikiText-2

The pilot generator retains natural WikiText language and creates *reference-conditioned continuation*, not conventional question answering. Each eligible source entry is divided into one to five earlier parts plus a tail. Earlier parts become URI-addressed references; the final 8–16 words of the tail become the prediction target. The visible prompt contains only reference handles and the neutral instruction `Continue the referenced WikiText passage:`. It therefore does not provide a local lexical prefix from which the target can be copied. Provenance records include the source entry, part boundaries and identifiers, candidate and intended references, tail span, token distance, part count, local-context sufficiency, and relation type.

That first generator labels all earlier parts as intended references and regenerates data from each run seed. We retain its measurements as a version-1 pilot. The controlled follow-up uses a version-2 dataset generated once with seed 1729 and shared across every architecture and model seed. It assigns candidate identifiers and order independently of relevance, marks one adjacent source part as the expected reference, and records source and split provenance. The fixed 512-example corpus yields 409/51/52 train/validation/test examples and a candidate-count-weighted test top-1 chance rate of 0.4202. This removes model-seed-dependent data pairing and permits actual selection metrics, although adjacency remains only a programmatic relevance label rather than human-verified causal evidence.

9 Training Objective and System

The training objective is standard autoregressive cross-entropy:

$$\mathcal{L} = -\sum_{t=1}^{T} \log p_\theta(x_{t+1} \mid x_{\le t}, C), \tag{32}$$

where C is the batch-specific PRA cache constructed from metadata. The training loop uses AdamW, optional learning-rate warmup, gradient accumulation, gradient clipping, optional mixed precision, checkpointing, early stopping, and TensorBoard/W&B/ClearML-compatible logging hooks.

The implemented PRA batch step is:

1. Move tensor fields to the target device.
2. For each example, build a fresh PRA cache from only that example's metadata without attaching it as a global cache.
3. Wrap the completed row caches in a batch-aware cache whose search maps query row b only to cache namespace b.
4. Attach that wrapper once and run one prompt forward on `input_ids` with shape $[B, T]$.
5. Compute cross-entropy with padding ignored.

Per-example cache isolation is semantically important. A previous batch-union implementation allowed each sequence to attend to references belonging to other examples and made within-batch shuffling ineffective. The corrected wrapper keeps the expensive Transformer prompt computation batched without flattening the logical address spaces. Reference-cache construction and routing are still row-wise; those are separate optimization targets from prompt-forward batching.

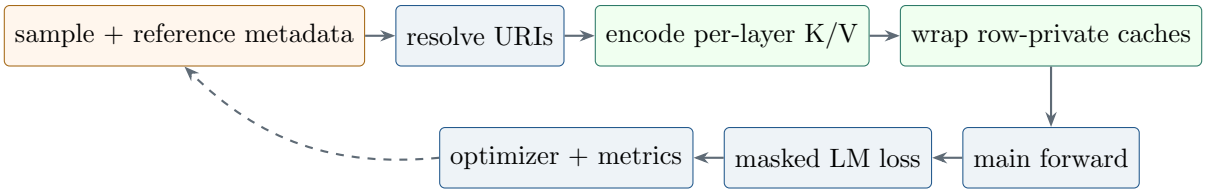

Figure 3: Implemented training/runtime pipeline. Row-private caches are constructed before one batched prompt forward; prompt labels are ignored in reference-conditioned loss.

9.1 Native Evaluation and Historical Two-Phase Optimization

The canonical path trains an ordinary SelfAttention language model, freezes its checkpoint, converts the existing Q/K/V and output projections, and evaluates native K/V without PRA training. The historical cross-attention follow-up instead evaluates three reference-task orders: (i) training from scratch; (ii) restoring the matching phase-1 checkpoint, freezing the base, zero-initializing $W_{O,\text{mem}}$, and optimizing only memory output projections; and (iii) restoring the checkpoint and directly fine-tuning all parameters jointly. Zero initialization makes the initial adapted branch an exact no-op. The frozen regime therefore preserves the pretrained local function by construction, while the measured plain-task reevaluation verifies that the implementation honors this constraint. The joint regime tests whether unconstrained adaptation is sufficient and quantifies catastrophic forgetting. These optimization findings apply to adapted cross-attention, not parameter-free native transport.

9.2 Execution-Time Accounting

The generic training engine records batch, epoch, and complete-run timing. CUDA is synchronized immediately before and after timed GPU regions so queued kernels are included. Reports contain training, validation, test, and wall-clock durations; processed tokens; sequences seen; optimizer steps; optimizer steps per second; and training tokens per second. Training duration sums synchronized batch regions, validation duration sums explicit validation passes, and wall-clock duration includes orchestration and test evaluation. These definitions should not be interchanged when comparing runtime cost.

10 Evaluation Metrics

The evaluation code reports both language-model and reference-behavior metrics:

- **Loss and perplexity**: autoregressive language-model quality.
- **Answer accuracy**: exact containment of the expected answer in generated output or decoded continuation.
- **Reference ranking**: actual selected-URI hit, recall, precision, F1, MRR, and NDCG at k, globally and per layer.
- **Chunk ranking**: selected-chunk hit, recall, precision, F1, MRR, NDCG, and optional source-span IoU against labeled chunks.
- **Expected anchor hit**: whether expected anchors appear in cached text or selected URI strings.

- **Expanded reference count**: average number of ready and recursively expanded references per example, with budget and failure events.
- **Available versus selected memory**: cache references, cache chunks/tokens, selected chunks, and actually materialized memory tokens.
- **Batching efficiency**: requested/actual bucket count, selected positions, allocated padded positions, and allocation ratio.
- **Full-context token count**: token cost of a full-context baseline.
- **Cache hit ratio**: fraction of examples whose expected references are represented in cache.
- **Latency and throughput**: wall-clock evaluation time, examples per second, and tokens per second.
- **Preservation loss**: plain WikiText-2 loss before and after reference-format fine-tuning.
- **Reference ablation loss**: answer-token loss with valid, disabled, shuffled, irrelevant, empty, and oracle reference memory.

These metrics are designed to test the PRA hypothesis directly. A useful PRA model should not merely achieve high answer accuracy; it should do so while selecting the right references and using fewer retrieved tokens than a full-context baseline.

The implemented ablation path supports disabled memory, shuffled references, irrelevant references, empty memory, oracle chunks, all selected chunks, full-reference materialization, and gist-only materialization. These are distinct interventions: cache size is not reported as selected-memory use, and a selected URI does not imply that all of its chunks were materialized. A valid-minus-disabled comparison tests whether the memory branch contributes. Valid versus shuffled or irrelevant is stricter because it asks whether the contribution depends on supplied content. Oracle chunks separate chunk routing from transport. The SelfAttention control is invariant across these conditions by construction.

10.1 Implementation Validation

The corrected mechanics are covered by focused tests rather than presented as model-quality evidence. Tests verify exact projected-key mean pooling; layer- and batch-specific routing; independent reference/chunk budgets; deterministic chunk provenance and overflow; explicit failure for unconfigured semantic chunking; optional summary fallback; GRU-gist gradients and checkpoint registration; recursive child-first build order and cycle safety; no-match equality with the local-only path; and dynamic-bucket output/gradient equivalence. A CUDA smoke test with two batch rows, different selected URIs, unequal memory lengths, and two buckets verifies finite forward/backward execution and independent selections. These checks establish implementation invariants only. The five-seed controlled study below then exercises the corrected path through training, counterfactual evaluation, retention checks, and synchronized timing; it does not by itself establish causal retrieval.

10.2 Required Systems Diagnostics

Task loss is insufficient to explain a memory system. Table 1 separates signals already emitted by the prototype from publication-grade diagnostics still required.

Table 1: Systems diagnostics and current implementation status.

Diagnostic	Measurement	Status
Reference selection	hit, recall, precision, F1, MRR, NDCG against relevant URIs	implemented globally and by layer
Chunk selection	ranking metrics and source-span IoU	implemented when labels are available
Attention behavior	routing scores, selected provenance, memory norms	structured trace implemented; heatmaps pending
Layer utilization	selected references/chunks/tokens and branch diagnostics	implemented per layer
Cache reuse	hit rate by URI/version and avoided encoding tokens	per-run hit only; persistent reuse pending
Latency	resolve/cache build, routing, materialization, memory attention, train/eval	cache and attention detail available; full resolver decomposition pending
Dynamic batching	selected versus allocated positions and bucket count	implemented with exact mode comparisons
GPU memory	peak allocated bytes and cache payload bytes	current allocation emitted; peak/decomposition pending
Failure cases	wrong URI, stale content, local insufficiency, poisoning, leakage	traces implemented; labeled taxonomy pending

Attention heatmaps should retain URI and token boundaries rather than concatenate memory into an anonymous axis. Layer utilization should distinguish a selected entry from an entry that receives meaningful attention mass. Cache reuse should count avoided encoding work, not merely dictionary hits. Failure reports should preserve prompt, target, candidate references, selected references, content versions, per-layer behavior, and the counterfactual outputs under disabled and shuffled conditions.

11 Experimental Protocol

The pilot compares three same-width architectures:

1. **SelfAttention**: four PyTorch causal SelfAttention layers.
2. **Full PRA**: four local-plus-memory PRAttention layers.
3. **Hybrid PRA**: two lower SelfAttention layers followed by two PRAttention layers.

All use four layers, four heads, width 256, context length 128, dropout 0.1, and one shared BPE tokenizer per study. The version-1 pilot uses seeds 1 and 7, batch size 2, learning rate 5×10^{-4}, 2,048 optimizer steps, and exactly 524,288 processed tokens per run. The corrected controlled follow-up uses paired seeds $\{1, 7, 21, 42, 87\}$, extends ordinary-language training to 4,096 steps and 1,048,576 tokens, fixes reference data and splitting with seed 1729, and compares 512-step scratch, frozen-reference-path, and joint reference training at learning rate 2×10^{-4}. All corrected runs execute with Python 3.10, PyTorch 2.12.1 with CUDA 12.6, and an NVIDIA GeForce GTX 950M.

The same-width parameter counts are 4,216,320 for SelfAttention, 4,479,488 for full PRA, and 4,347,904 for hybrid PRA. Full and hybrid PRA therefore contain 6.2% and 3.1% more parameters than SelfAttention. In phase 2, the PRA trainable counts are 263,168 and 131,584 because only four or two memory output projections are optimized; the SelfAttention control fine-tunes all 4,216,320 parameters. These differences delimit the claims that can be drawn from the pilot.

Table 2: Current pilot hyperparameters. Phase-2 processed tokens vary slightly with generated example lengths.

Setting	Phase 1: plain WikiText-2	Phase 2: references
Tokenizer	shared BPE, vocabulary 2,000	restored phase-1 BPE
Model	4 layers, 4 heads, width 256	matching phase-1 checkpoint
Context/batch	128 / 2	128 / 2
Optimizer budget	2,048 steps; 524,288 tokens	512 steps; mean 46,301.5 tokens
Learning rate	5×10^{-4}	2×10^{-4}
Dropout	0.1	0.1
References	absent	1–5 parts; top-$k = 5$
Threshold/α	inactive	-1.0 / 1.0
Trainable parameters	all	all SA; memory output only for PRA
Seeds	1, 7	1, 7

The parameter-matched SelfAttention controls retain width 256 and increase each four-layer FFN hidden width from 1,024 to 1,152 or 1,088. The resulting models contain 4,478,976 and 4,347,648 parameters, respectively: 512 fewer than full PRA and 256 fewer than hybrid PRA. This isolates most of the attention-branch capacity difference without changing layer count, head count, or representation width.

The evaluated ablations are:

- valid per-example references;
- a cleared and explicitly disabled PRA path;
- references deterministically shuffled across examples;
- plain WikiText-2 reevaluation after phase 2.

The corrected follow-up evaluates valid, disabled, shuffled, irrelevant, empty, and oracle-reference conditions plus parameter-matched controls in cross-attention mode. Native evaluation begins instead with `SA-full`, `SA-tail`, `PRA-native-all`, `PRA-native-oracle`, `PRA-native-disabled`, and `PRA-native-shuffled`. The report computes $\Delta_{\text{transport}} = L_{\text{native-all}} - L_{\text{SA-full}}$, $\Delta_{\text{sparse}} = L_{\text{native-oracle}} - L_{\text{native-all}}$, $\Delta_{\text{routing}} = L_{\text{native-routed}} - L_{\text{native-oracle}}$, $\Delta_{\text{memory}} = L_{\text{SA-tail}} - L_{\text{native-oracle}}$, and $\Delta_{\text{causal}} = L_{\text{native-shuffled}} - L_{\text{native-oracle}}$. Together, the first three gaps decompose the path from dense SelfAttention to all-memory transport, sparse oracle materialization, and routed selection. We also report Recovered Context Benefit for condition c,

$$\text{RCB}_c = \frac{L_{\text{SA-tail}} - L_c}{L_{\text{SA-tail}} - L_{\text{SA-full}}}. \tag{33}$$

RCB is not clipped: zero means no dense-context benefit is recovered, one means all of it is recovered, and negative values mean memory is harmful relative to the tail. It is marked undefined when the full-versus-tail denominator is numerically negligible. Active fraction is ($T_{\text{local}} +$

$T_{\text{retrieved}})/(T_{\text{local}} + T_{\text{displaced}})$, where routing gists and index vectors are recorded separately rather than counted as token K/V. The raw per-example records additionally contain target identity, selected URI/chunk sets, hit and recall signals, K/V transfer bytes, routing/materialization/attention time, and peak CUDA allocation. Reports aggregate per example, seed, and split with means, population standard deviations, medians, and approximate 95% intervals.

11.1 Native-KV Experimental Progression

The first gate converts one trained SelfAttention checkpoint and verifies disabled-memory logit equivalence. The second extracts historical K/V while processing prefix A in its original left context and reinserts all of it while processing tail B; this must match the tail of full attention with equivalent positions and masks. Independent chunk re-encoding is reported separately because reset positions and missing left context make it a different intervention. Only after all-K/V transport passes should an oracle active K/V budget frontier be measured, followed by routed and shuffled controls.

The fixed-target WikiText generator supports split counts $\{2, 3, 5, 8, 16, 32, 64\}$. For a given source and seed, it keeps the recent direct tail, answer, target identifier, and evaluation mask unchanged. Only the boundaries inside displaced prefix history vary; metadata-only implicit references avoid adding visible handle tokens. This corrects the earlier split-2/split-5 design, whose target and processed-token count changed with split count. The synthetic and evidence-anchored natural-text probes below use this invariant writer and verify the fixed target identifier before evaluation.

11.2 Publication-Grade Experiment Matrix

The current pilot is the first row of a larger preregistered-style matrix. Table 3 marks required comparisons without assigning unmeasured values.

Table 3: Required experiment matrix after the controlled follow-up. "Pending" means no empirical claim is made in this paper.

Axis	Required comparison	Status
Reproducibility	seeds $\{1, 7, 21, 42, 87\}$ with intervals/effect sizes	five paired seeds complete
Capacity	tiny and small SA backbones, then weight-preserving PRA conversion	five seeds through 256 units
Reference causality	valid, disabled, shuffled, irrelevant, empty, oracle chunks	corrected URI-level smoke complete; chunk-level study pending
Cache budget	entries/tokens/bytes; reuse on/off; eviction policies	pending
Memory strength	α sweep including zero and learned gate	$\alpha = 1$ only
Selection	reference top-k and multi-gist routing representation	five-seed top-k and staged gist sweeps complete
Fragmentation	independent, overlap, block slicing, native historical slicing	split-64 screen and five-seed confirmation complete
Transport	native all/oracle/routed versus adapted cross-attention	synthetic, HotpotQA-derived, and QASPER-derived native sweeps complete
Adaptation	scratch, frozen branch, LoRA, adapters, joint fine-tuning	scratch/frozen/joint complete
Placement	full PRA, upper-layer hybrid, layer-count sweep	full and 2+2 hybrid complete
Difficulty	distance, candidate count, local sufficiency, recursion depth	recursive runtime present; experiments pending

The five-seed comparison uses a fixed tokenizer and split, equal processed-token budgets, and paired seeds. Future cache and selector sweeps should initialize from the same checkpoints. Oracle references isolate memory transport from selection; irrelevant and empty controls distinguish content use from generic branch activation. Native K/V injection must account for positional encoding and causal semantics rather than simply concatenate tensors. LoRA, adapters, and joint fine-tuning should be matched by trainable parameter count or reported as separate adaptation regimes.

12 Fixed-Source Native-KV Synthetic Result

This first experiment asks whether native K/V can carry a causally necessary value when the correct memory is supplied, before semantic routing or natural-language competence is allowed to confound the answer.

The first native quality gate uses 2,048 training examples and a disjoint 64-example evaluation set. Every example contains the same 63 source-unit structure, a fixed historical marker whose random binary value is the target, and a one-token local query with no answer information. The seven conditions partition the identical source into 1, 2, 4, 7, 15, 31, or 63 references. Thus source text, target, model weights, and local prompt are invariant; only reference boundaries change. Five paired seeds train a two-layer, width-64 SelfAttention model for 100 optimizer steps on full

context. Each checkpoint is then converted weight-for-weight to native PRA. No PRA transport or selector parameter is trained. The fixed marker position deliberately isolates transport from semantic search; random-position and learned-routing tasks are harder follow-ups.

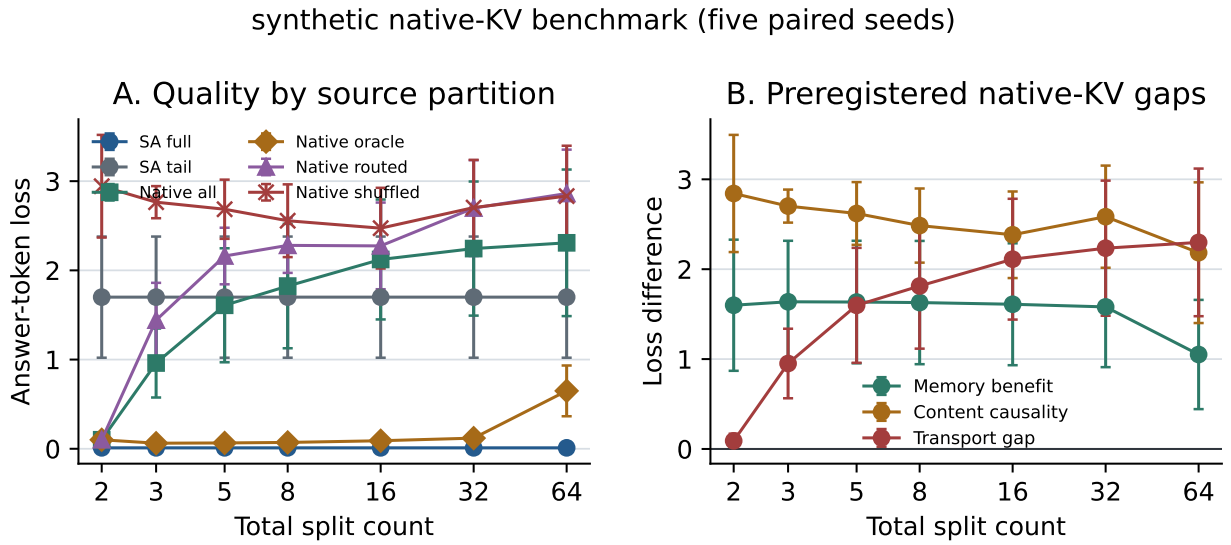

Figure 4: Five-seed fixed-source synthetic native-KV result. Full-context SelfAttention solves the dependency while the local tail is at chance. Oracle memory preserves the required content through 32 splits and sharply separates from shuffled content. Native-all and unsupervised routed conditions degrade as the same source is fragmented, distinguishing successful sparse transport from unresolved cache-composition and selection behavior. Error bars are population standard deviations.

Full-context SelfAttention obtains mean loss 0.0105 and 100% accuracy, whereas tail-only evaluation obtains loss 1.6999 and 50.6% accuracy. At split counts 3, 5, 8, 16, and 32, oracle native loss is respectively 0.0624, 0.0653, 0.0711, 0.0900, and 0.1194, with 100% accuracy in every case. Shuffled losses over the same range are 2.7645, 2.6853, 2.5565, 2.4730, and 2.7038; the resulting content-causality gaps are 2.7021, 2.6200, 2.4854, 2.3831, and 2.5844. This is direct evidence that the oracle result depends on the supplied value rather than generic memory activation.

Historical diagnostic result, not the final PRA scaling limit. The split-64 condition below independently encoded every small reference fragment and restarted its positions. Later experiments encode a larger causal region and slice its native K/V into smaller routing chunks; that contextual/native-KV path removes this failure through 256 addressable units. The old result is retained because it revealed why **encoding granularity and routing granularity must differ**.

Under the historical independent-encoding condition, at 64 splits oracle loss rises to 0.6490 and accuracy falls to 70.6%. Native-all accuracy declines from 100% at split 2 to 71.9% at split 3, 59.1% at split 5, and approximately chance by split 16. The routed condition follows the same failure pattern and is 48.4% at split 32. Because positions restart at every fragment and selected all-memory payloads are not a causally encoded prefix, practical native-all is not equivalent to exact historical-K/V reinsertion. This experiment validates sparse oracle transport under moderate independent fragmentation; it does not establish a 64-way limit for the corrected architecture or learned routing.

13 Evidence-Anchored Natural-Text Native-KV Probes

The next experiment asks whether the same transport result survives natural-document context and distractors, while retaining a controlled target whose dependency is known.

The next gate retains the controlled target while replacing character-level distractors with natural documents from HotpotQA and QASPER [19, 5]. These are deliberately transport probes, not benchmark QA scores. For HotpotQA we use balanced yes/no questions, the supplied supporting-fact sentences, and distractor context. For QASPER we use balanced yes/no annotations and their human evidence spans from the official v0.3 corpus. Each example contains 63 fixed source units. The first two units are an explicit `AnswerCode yes|no` anchor; the remaining units contain evidence and sampled document words. The local tail asks only for that code. The original question, evidence, paper/document identity, and source metadata remain in the machine-readable record.

This construction separates two questions that a tiny model otherwise confounds. It does not ask whether a two-layer decoder trained from scratch can perform multi-hop or scientific-document QA. It asks whether a normally trained SelfAttention checkpoint can use a causally necessary token in natural-text context, whether sparse native K/V recovers that benefit after the source is displaced, and whether gist routing finds the known anchor. A direct HotpotQA pilot without the anchor failed this gate: held-out dense-context loss was worse than tail-only loss, making RCB uninterpretable. We therefore reject that pilot rather than attribute language-model failure to PRA.

Training and evaluation examples come from disjoint official splits. Across total split counts $\{2, 3, 5, 8, 16, 32, 64\}$, source text, local tail, answer token, tokenizer, checkpoint, and example order are identical; only reference boundaries vary. We report the same five paired seeds $\{1, 7, 21, 42, 87\}$ and the same native conditions as in the synthetic gate. No PRA transport or routing parameter is trained after the full-context SelfAttention checkpoint is frozen and converted.

13.1 HotpotQA-Derived Result

The HotpotQA-derived probe uses 4,000 balanced training examples and 64 examples from the disjoint validation split. A two-layer, width-128 decoder is trained for 150 optimizer steps with an 8,000-token BPE vocabulary. Table 4 reports five-seed means. Dense full context solves the target with loss 0.0027, while tail-only loss is 1.7038. The denominator required by RCB is therefore large and positive rather than an artifact of a low-dependency target.

Table 4: HotpotQA-derived answer-code transport probe, five-seed means. RCB is recovered context benefit; active is the oracle active token-K/V fraction.

Split	SA full	SA tail	Native all	Oracle	Routed	Shuffled	Oracle RCB	Routed RCB	Active
2	.0027	1.7038	.0029	.0029	.0029	4.0042	1.000	1.000	1.000
3	.0027	1.7038	.0249	.0035	.3835	2.5983	.999	.807	.524
5	.0027	1.7038	.0408	.0051	.5944	2.1552	.997	.653	.284
8	.0027	1.7038	.0432	.0056	.6168	2.0606	.997	.639	.202
16	.0027	1.7038	.0786	.0058	.6179	1.9974	.996	.641	.129
32	.0027	1.7038	.1213	.0060	.6812	2.2251	.996	.634	.092
64	.0027	1.7038	.9541	.8045	.9239	1.2470	.546	.405	.092

The decomposition localizes the result. At split 32 the mean transport gap is 0.119, the sparse gap is -0.115, and the routing gap is 0.675. Independent all-memory composition has accumulated partition cost, while selecting the two anchor units removes most of that cost; routing, not oracle

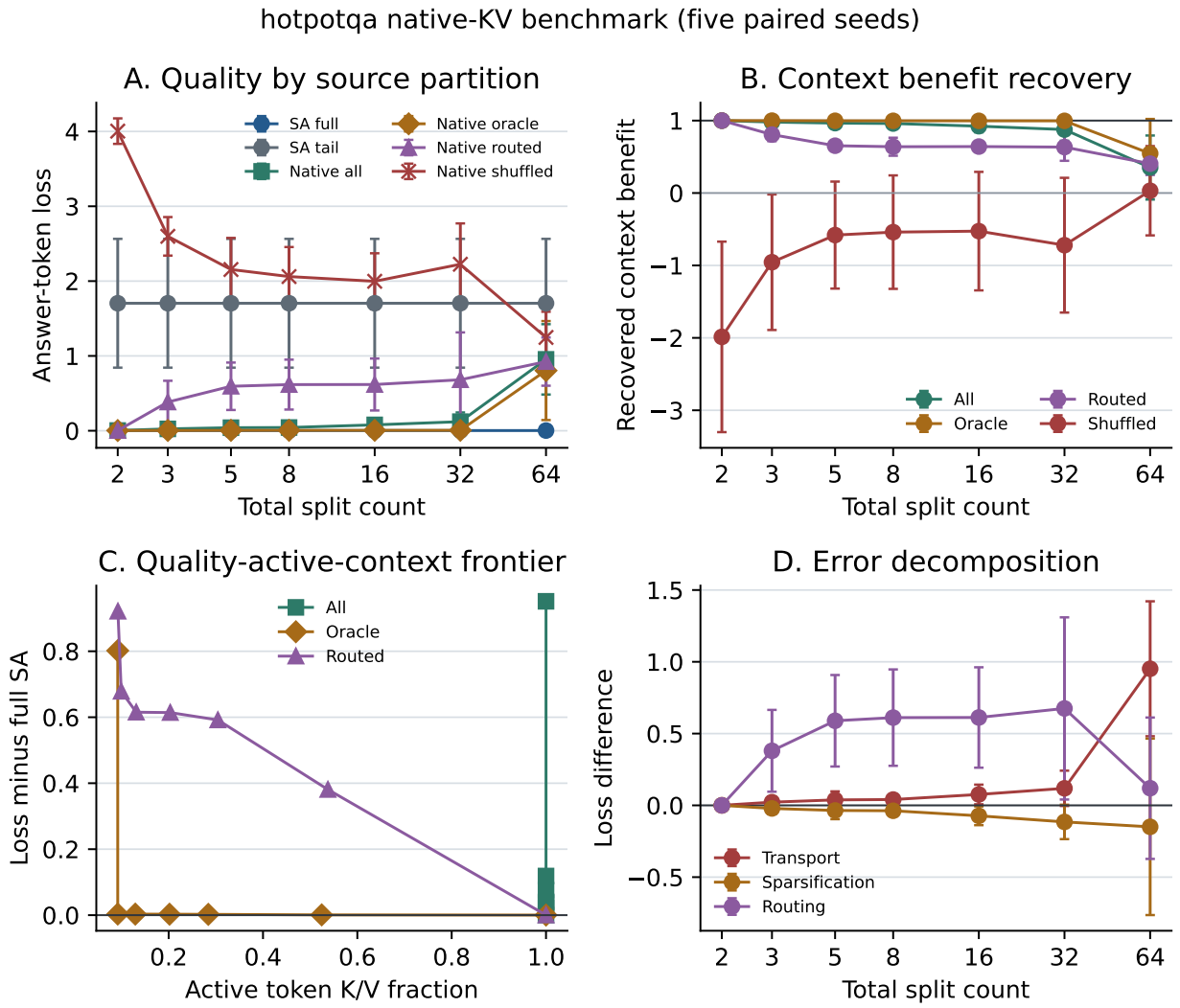

Figure 5: HotpotQA-derived quality, RCB, active-context frontier, and error decomposition. Oracle native K/V tracks dense context through split 32 while routed selection leaves a substantial oracle-to-routed gap. Error bars are population standard deviations over five paired seeds.

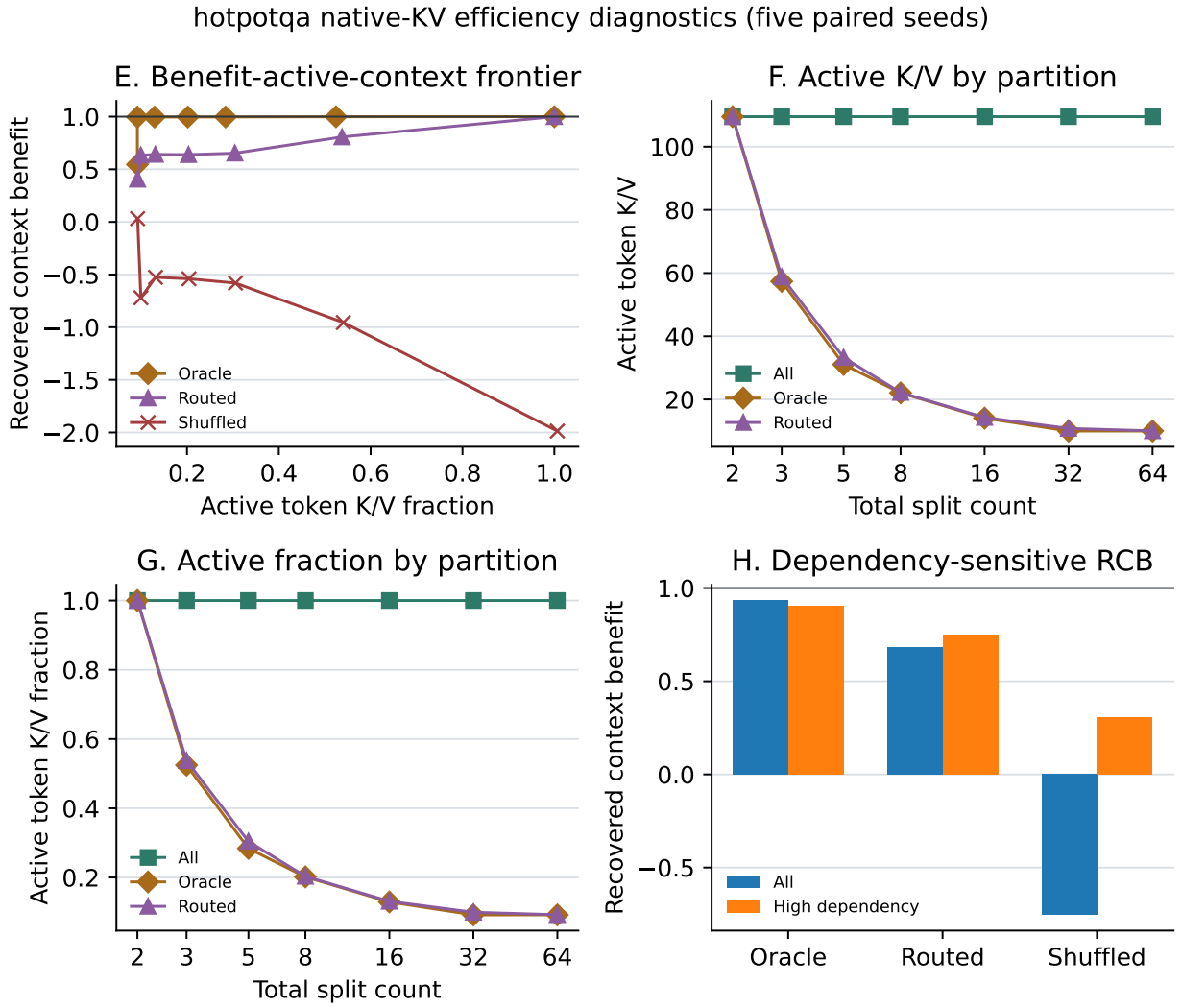

Figure 6: HotpotQA-derived active-K/V diagnostics. Through split 32, oracle RCB remains near one as active fraction falls to 0.092. The high-dependency stratum is formed from data-driven dependency-gain tertiles, not a hand-selected threshold.

transport, is the dominant remaining error. The oracle content-causality gap at split 32 is 2.219 loss units. Across all splits, the high-dependency tertile has mean oracle RCB 0.903, routed RCB 0.749, and shuffled RCB 0.309. These per-example ratios are more variable than the aggregate-loss ratios and are reported with medians in the machine-readable artifact.

Split 64 is a clear boundary. Oracle loss rises to 0.8045 and oracle RCB falls to 0.546; individual seeds range from near-dense recovery to losses above the tail baseline. This is not an active-budget increase—the mean active fraction remains 0.092—but a fine-fragment encoding/composition failure. The same condition also reduces the all-memory versus oracle contrast, so the 64-way point should not be read as evidence that routing has suddenly improved.

The timing fields are diagnostic rather than a hardware benchmark: these runs use a tiny CPU model and a Python cache builder. At split 32, oracle materializes about 10 active tokens and transfers 4,096 K/V bytes per example on average, versus about 110 active tokens and 207,872 bytes for all-memory. Mean measured example time is 10.7 ms for oracle and 853 ms for all-memory; at split 64 it is 12.5 ms versus 1.70 s. These numbers include prototype overhead and do not establish production speedup, but they verify that the quality-active-context frontier corresponds to measured materialization and transfer reductions rather than a label-only budget.

13.2 QASPER-Derived Result

The QASPER-derived probe repeats the protocol on scientific papers and human-annotated evidence spans. It uses 200 balanced training probes and 64 probes from the disjoint development split, the same two-layer width-128 architecture and 150-step schedule, and a BPE vocabulary learned only from the training corpus. Table 5 reports five-seed means.

Table 5: QASPER-derived answer-code transport probe, five-seed means. Active is the oracle active token-K/V fraction.

Split	SA full	SA tail	Native all	Oracle	Routed	Shuffled	Oracle RCB	Routed RCB	Active
2	.0026	1.8545	.0035	.0035	.0035	4.8820	.999	.999	1.000
3	.0026	1.8545	.0610	.0036	.2977	3.4029	.999	.790	.525
5	.0026	1.8545	.0925	.0037	.4924	2.6978	.999	.661	.281
8	.0026	1.8545	.1267	.0037	.6078	2.6359	.999	.572	.192
16	.0026	1.8545	.1769	.0037	.6640	2.2348	.999	.542	.117
32	.0026	1.8545	.1612	.0039	.5550	2.7351	.999	.602	.088
64	.0026	1.8545	.8835	.5047	1.0982	1.1819	.748	.338	.088

QASPER reproduces the HotpotQA-derived pattern with a different source distribution. At split 32, the transport gap is 0.159, sparse gap is -0.157, and routing gap is 0.551. The oracle content-causality gap is 2.731. High-dependency examples have mean oracle, routed, and shuffled RCB of 0.968, 0.823, and 0.391, respectively. At split 64, oracle loss rises to 0.5047 and oracle RCB falls to 0.748, while routed RCB falls to 0.338. The failure is milder than HotpotQA's mean oracle degradation but remains strongly seed-dependent.

At split 32, oracle activates about 10 tokens and transfers 4,096 K/V bytes per example, versus 114 tokens and 217,920 bytes for all-memory. Mean prototype times are 22.1 ms and 604 ms, respectively; at split 64 they are 25.2 ms and 1.26 s. Because all quality runs use CPU and a Python reference runtime, these values validate accounting and direction, not optimized systems performance. Together, the two natural-text probes support sparse native transport and expose a repeatable routing gap. They still do not establish unrestricted HotpotQA or QASPER question answering, because the answer-code anchor makes the causal dependency controlled and explicit.

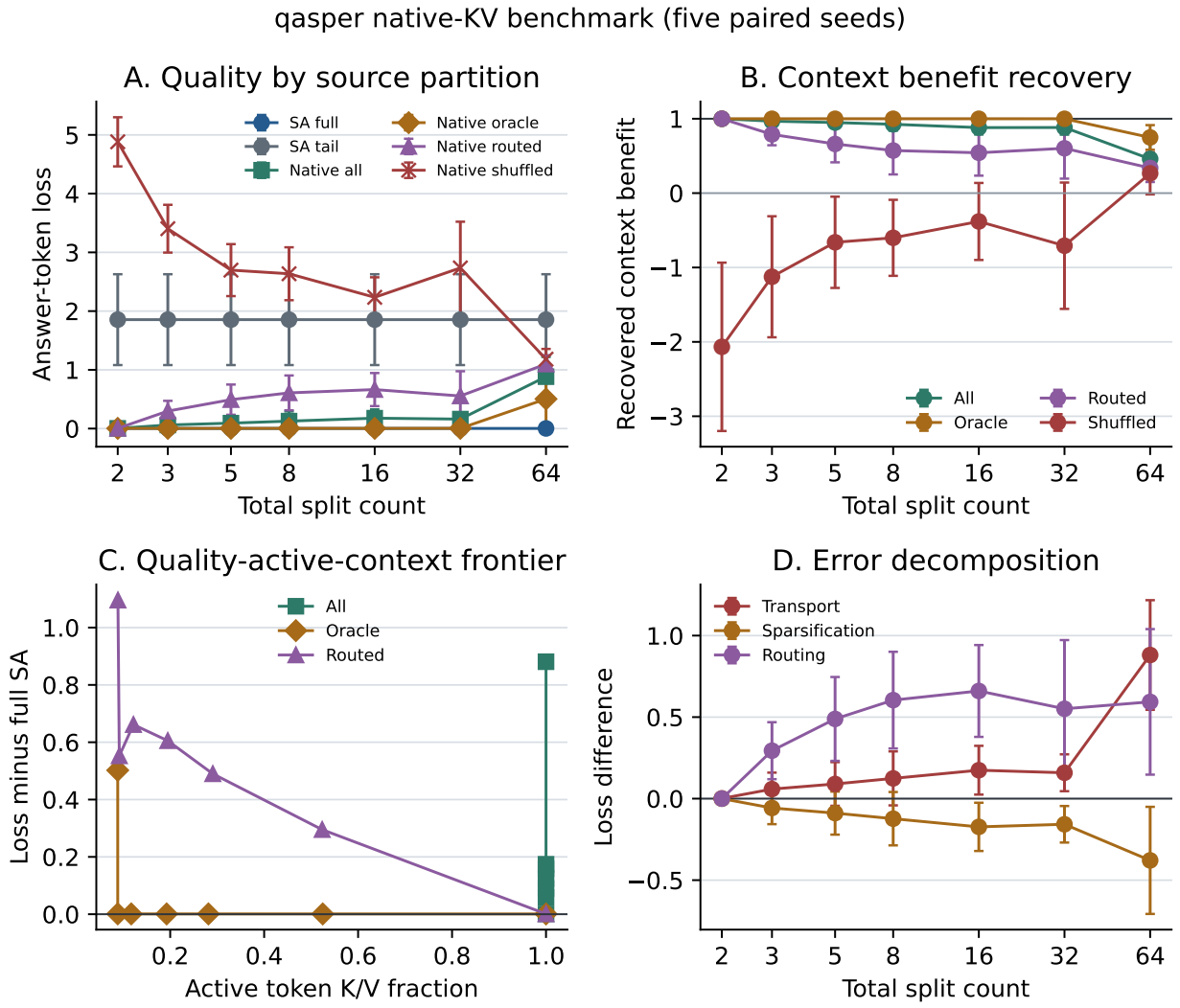

Figure 7: QASPER-derived quality and error decomposition over five paired seeds. Oracle native K/V preserves dense-context quality through split 32, while all-memory composition and routed selection accumulate distinct errors.

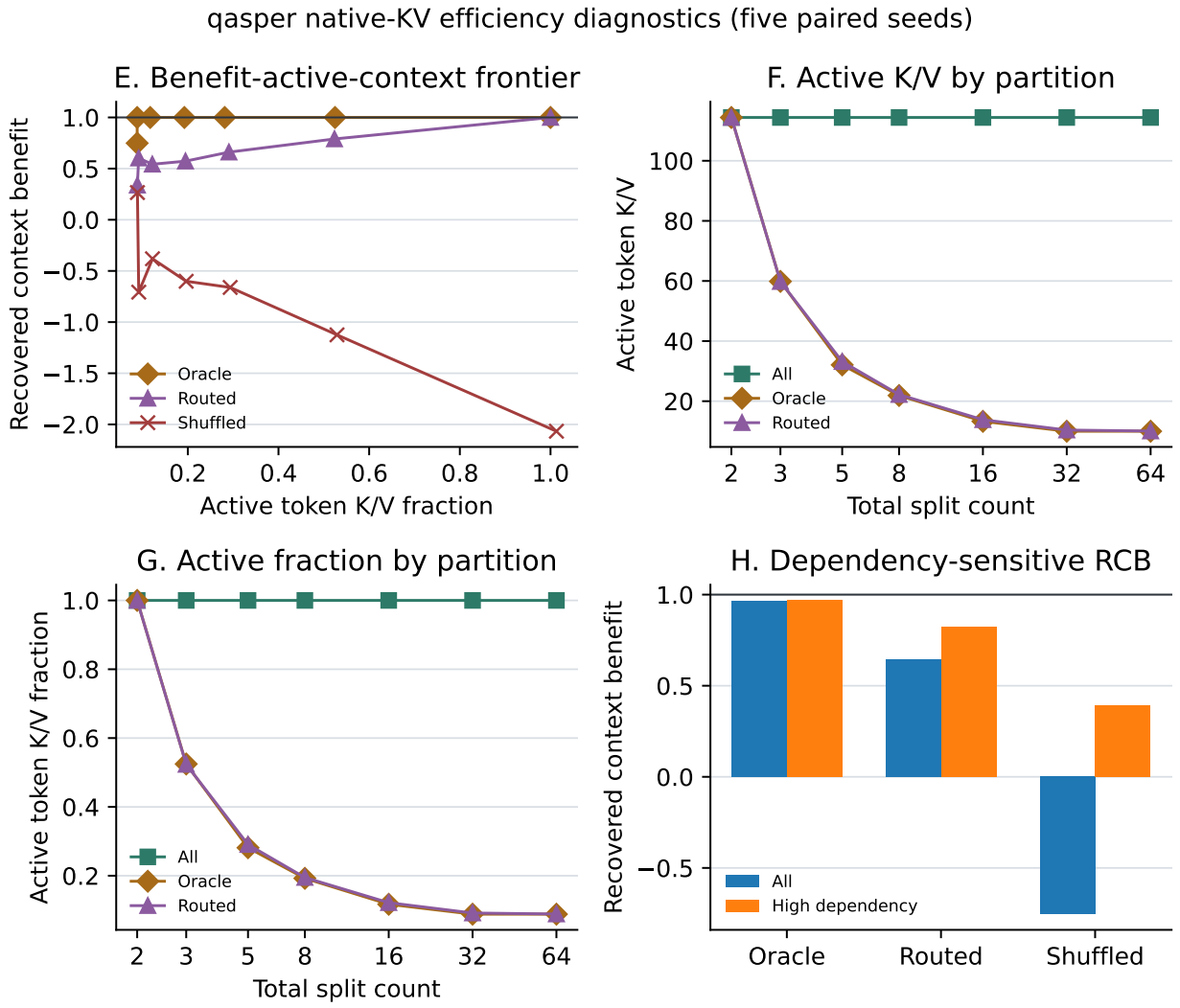

Figure 8: QASPER-derived active-K/V diagnostics. Oracle RCB remains 0.999 as active fraction falls to 0.088 through split 32; the 64-way point again marks a fragmentation limit.

14 Routing, Fragmentation, and Scale Sensitivity

The preceding fixed-source results intentionally used a weak baseline: one mean gist per chunk, one selected reference, one selected chunk per reference, and independent reference encoding. We next varied retrieval breadth, gist representation, encoding context, and backbone capacity in that order. Every reported aggregate uses paired seeds $\{1, 7, 21, 42, 87\}$ unless explicitly identified as a one-seed screen. The search was staged rather than Cartesian: a top-k frontier preceded a multi-gist screen; promising encoding strategies were then confirmed at split 64; finally the complete tiny-model top-k frontier and one selected small-model setting were carried to 128 and 256 units. Machine-readable manifests preserve this selection protocol.

14.1 Retrieval Breadth and Gist Representation

At split 32, increasing reference top-k improves routed RCB on both domains, but costs more active K/V. HotpotQA-derived RCB rises from 0.634 at $k = 1$ to 0.777 at $k = 8$, while active fraction rises from 0.099 to 0.295. QASPER-derived RCB rises from 0.602 to 0.874 while active fraction rises from 0.092 to 0.260. Neither reaches the preregistered RCB$\geq$ 0.90 and active$\leq$ 0.20 target. At split 64, breadth alone does not repair the failure: HotpotQA-derived RCB remains near 0.40 through $k = 16$, and QASPER-derived RCB reaches only 0.435. Complete two-target coverage is still 0.078 and 0.138 at $k = 16$.

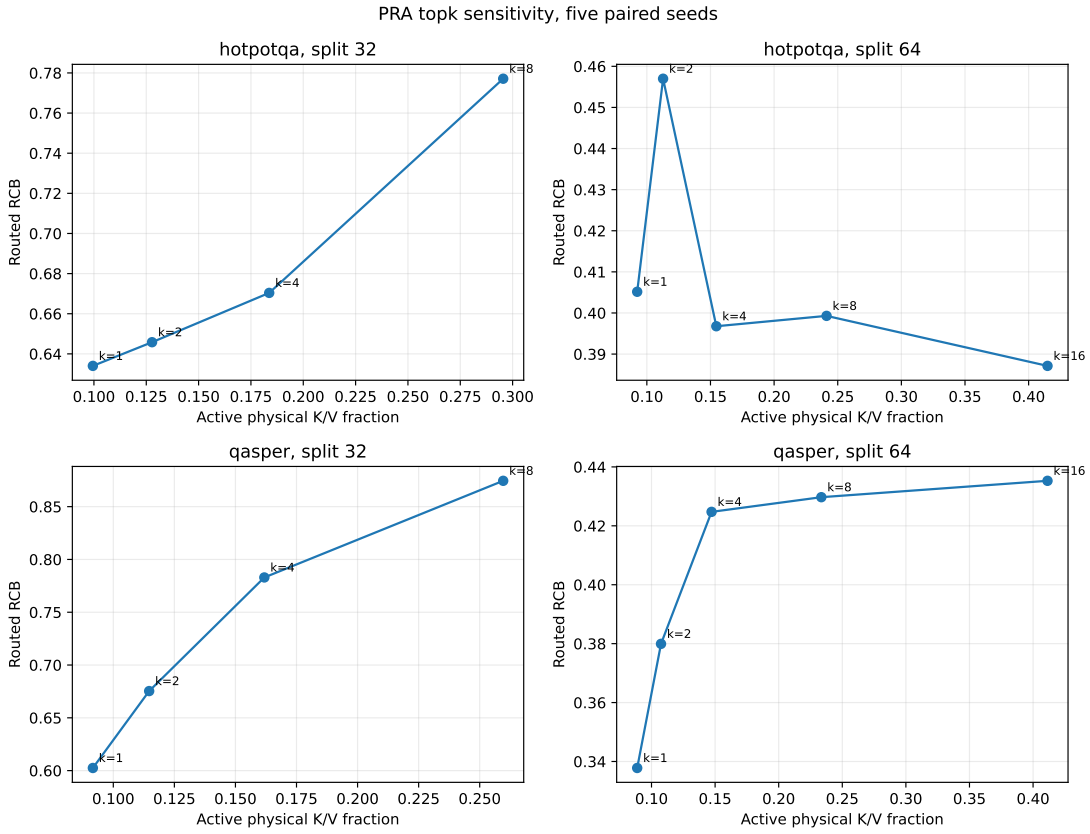

Figure 9: Five-seed retrieval-breadth frontier under independent encoding. More references improve split-32 quality but increase transported K/V; split-64 quality does not recover, indicating that ranking breadth is not the only bottleneck.

The staged gist screen compares mean, last-token, prototype, k-means, SOM, and hybrid representations with one, two, or four gists where meaningful. No multi-gist mode is a robust universal winner. At split 32 and $k = 4$, one-gist SOM raises QASPER-derived routing MRR from 0.315 for mean pooling to 0.395, but HotpotQA-derived MRR is 0.332 and RCB falls from 0.670 to 0.634. Hybrid four-gist routing helps one QASPER seed but collapses on HotpotQA-derived text. The result argues against treating more prototypes as a free quality improvement; they increase comparisons and remain representation-sensitive.

14.2 Fragmentation Decomposition

This experiment tests whether the high-split failure is primarily insufficient routing breadth or a contextualization failure caused by independently encoding tiny fragments.

The split-64 oracle decline is primarily an encoding-context failure. Independent microchunks reset both left context and positions. Encoding eight consecutive references together restores five-seed annotated-oracle RCB from 0.546 to 0.996 on HotpotQA-derived text and from 0.748 to 0.999 on QASPER-derived text. Keeping global source positions raises routed RCB from 0.706 to 0.865 and from 0.799 to 0.993. Full historical native slicing is best or tied: routed RCB is 0.876 and 0.994 at approximately 10% active K/V.

One-seed overlap screens at block size eight test 0.05, 0.10, and 0.20 left-context fractions. They recover oracle quality because the enclosing block already restores context, but do not show a stable advantage over nonoverlapping block/native slicing. Encoding duplication is 1.0 for the five-seed selected conditions because stored routing regions remain nonoverlapping. We therefore retain overlap as a boundary diagnostic, not as the selected scale strategy. The stronger conclusion is that addressable granularity need not equal encoding granularity.

Takeaway. The high-split failure was primarily an encoding-context artifact, not evidence that PRA could not route among many named units. Larger contextual encoding blocks can produce many smaller routable native-K/V slices.

14.3 Meaningful 64–256 Unit Scaling

Having repaired fragmentation, this experiment asks whether selected K/V can remain useful as both source length and the number of addressable units grow, rather than merely slicing the same fixed source more finely.

The scale study grows source length with addressability instead of repartitioning 63 fixed units. The nested evaluation sources contain 63, 127, and 255 units; mean accessible BPE lengths grow from 110 to 440 for HotpotQA-derived text and from 101 to 388 for QASPER-derived text. The SelfAttention backbone is trained only on full 255-unit context, then converted without PRA training. Native historical encoding runs once, slices K/V into one URI per source unit, preserves global source positions, and continues tail positions after the source. Each seed evaluates 32 held-out examples. Tiny backbones have 2.2–2.4M parameters and train for 200 steps; small backbones have 6.6–6.9M parameters and train for 300 steps. Every final checkpoint reaches 100% answer-token validation accuracy. The tiny tier evaluates $k \in \{2, 4, 8, 16, 32\}$; the small tier confirms the selected $k = 8$ setting.

At $k = 8$, active fraction approximately halves whenever source size doubles: tiny-model HotpotQA-derived active fraction is 0.190, 0.095, and 0.047, while QASPER-derived values are 0.196, 0.100, and 0.051. Retrieved physical K/V remains about 11–13 tokens. This is the desired sparsity law for a fixed retrieval budget: accessible context grows roughly fourfold while active memory remains nearly constant.

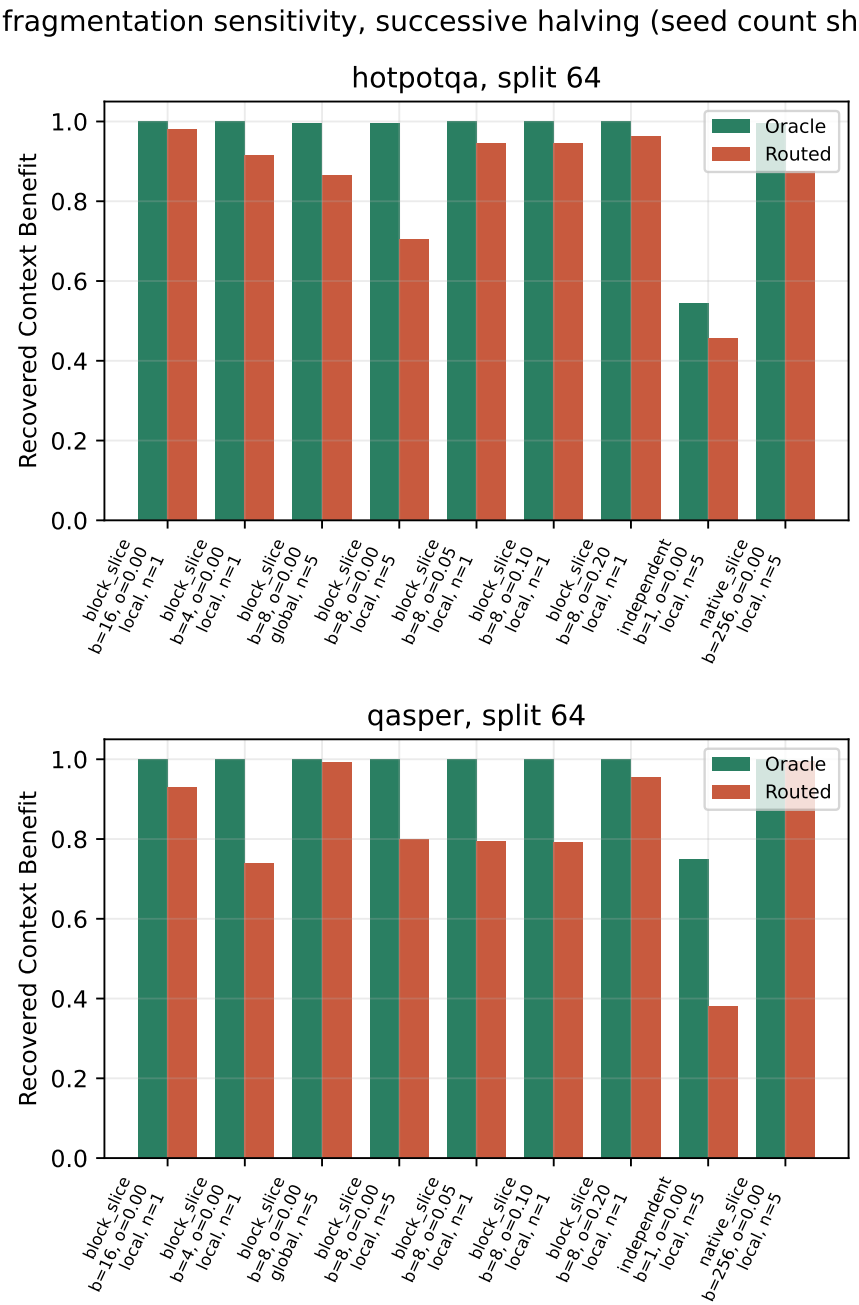

Figure 10: Split-64 fragmentation decomposition. Labels report the one-seed block-size/overlap screen and five-seed confirmations for independent, block-8 local/global, and native-slice conditions. Larger causal encoding context restores transport quality before routing is tuned.

Table 6: Meaningful context scaling at routed $k = 8$, five-seed means. "Active" is physical local-plus-retrieved K/V divided by accessible local-plus-historical tokens. MRR scores annotated target URIs.

Probe	Tier	Units	SA full	SA tail	Oracle RCB	Routed RCB	Active
HotpotQA	tiny	64	.00275	2.5503	.940	.999	.190
HotpotQA	tiny	128	.00256	2.5503	.981	1.000	.095
HotpotQA	tiny	256	.00255	2.5503	.968	1.000	.047
HotpotQA	small	64	.00027	3.0718	1.000	1.000	.179
HotpotQA	small	128	.00026	3.0718	1.000	1.000	.089
HotpotQA	small	256	.00026	3.0718	1.000	1.000	.044
QASPER	tiny	64	.00240	2.4694	.998	1.000	.196
QASPER	tiny	128	.00245	2.4694	.999	.981	.100
QASPER	tiny	256	.00239	2.4694	.998	.908	.051
QASPER	small	64	.00027	3.3264	.988	1.000	.187
QASPER	small	128	.00026	3.3264	.987	1.000	.097
QASPER	small	256	.00026	3.3264	.990	1.000	.049

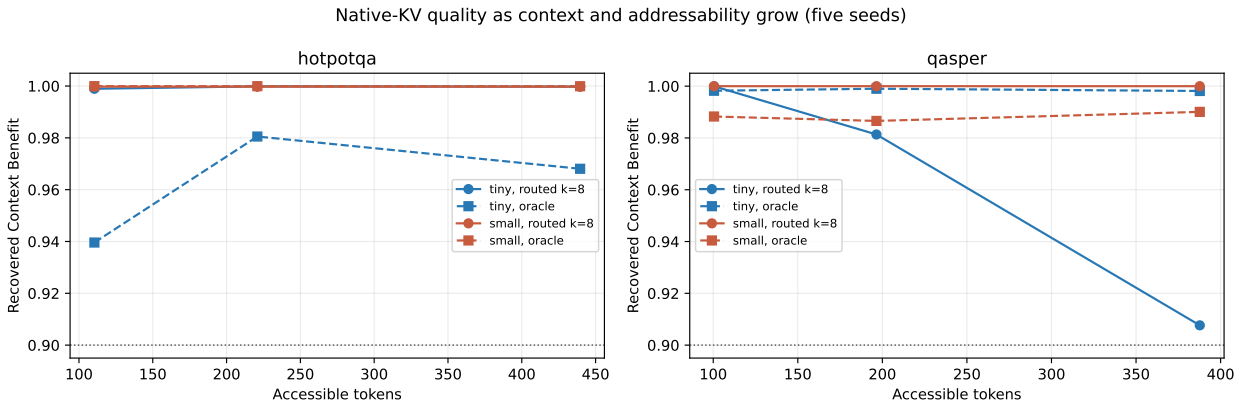

Figure 11: Recovered context benefit as source length and addressability grow. Native historical slicing keeps annotated-oracle quality healthy. The tiny QASPER-derived router degrades at 128/256, while the small backbone restores routed quality at the same $k = 8$ budget.

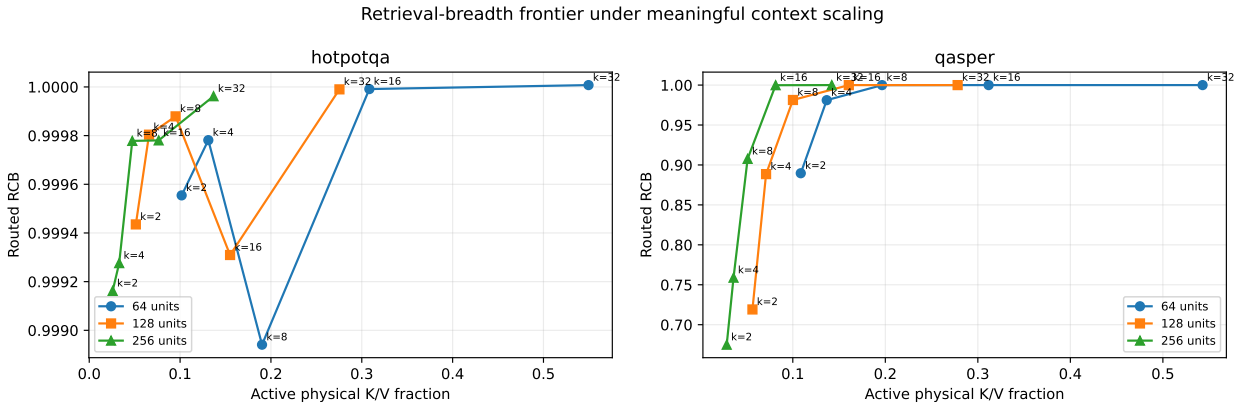

Figure 12: Tiny-backbone top-k frontier. QASPER-derived text requires increasing breadth as addressability grows: at 256 units, routed RCB rises from 0.675 at $k = 2$ to 0.908 at $k = 8$, 1.000 at $k = 16$, and 1.000 at $k = 32$. HotpotQA-derived loss is nearly insensitive because the answer-bearing native state is consistently selected.

Table 7 states the same result as avoided K/V. The uniform estimate assumes equal-length chunks and complete token-level materialization of the eight selected chunks. The measured reference-only range instead divides retrieved physical K/V by all unique source K/V, so it includes variation in BPE chunk length. The total working-set range additionally retains the fixed local prompt K/V. Under the experiment's convention, a nominal N-way split contains $N - 1$ addressable references and one local tail. If all N chunks were addressable, the corresponding uniform savings would be 75.0%, 87.5%, 93.75%, and 96.88% for 32, 64, 128, and 256 chunks.

Table 7: K/V materialization savings at routed $k = 8$. The 32-way row is an equal-chunk estimate; measured ranges come from the five-seed 64/128/256 scale runs across both datasets and model tiers. "Total" includes local prompt K/V.

Nominal splits	References	Uniform reference K/V avoided	Measured reference K/V avoided	Measured total K/V avoided
32	31	74.2%	—	—
64	63	87.3%	87.4–88.6%	80.4–82.1%
128	127	93.7%	93.9–94.5%	90.0–91.1%
256	255	96.9%	96.9–97.4%	94.9–95.6%

Capacity changes routing geometry as well as dense loss. At 256 QASPER-derived units, the small tier raises MRR from 0.240 to 0.443 and target fraction covered at $k = 8$ from 0.388 to 0.759. HotpotQA-derived MRR rises from 0.325 to 0.523 and coverage from 0.378 to 0.700. This is measured evidence that the tiny backbone understated routing quality. It is not evidence that scale alone solves general retrieval: the task has an explicit answer-code anchor, and no router parameters are trained.

Annotated oracle is not a strict loss upper bound in this construction. Target labels include answer and evidence-bearing URI regions. Under causal historical encoding, a later answer-bearing URI's hidden K/V may already contain earlier `AnswerCode` context. Routing that one sufficient state can outperform materializing every annotated target, whose extra K/V changes softmax composition. This explains routed RCB slightly above annotated-oracle RCB and the coexistence

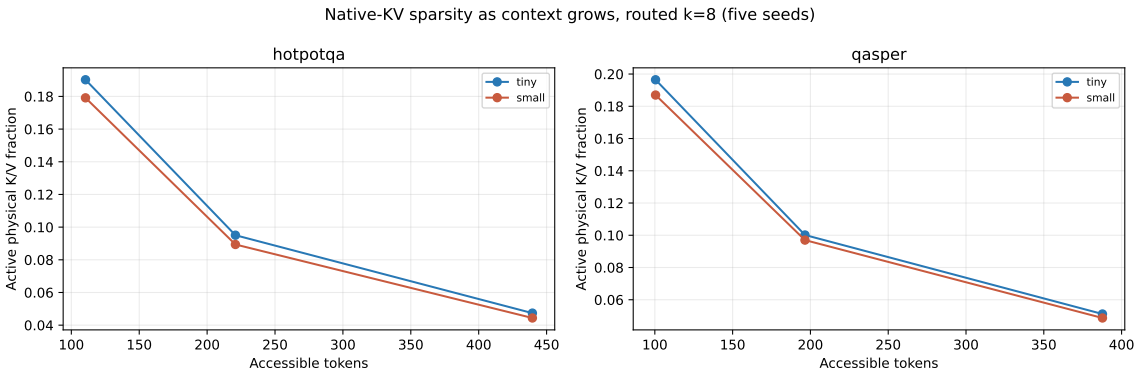

Figure 13: Physical active fraction at routed $k = 8$. Retrieved token count remains nearly constant while accessible context grows, so active fraction falls approximately inversely with source size.

of near-perfect answer loss with incomplete multi-target coverage. The result demonstrates causal transport of a sufficient native state, not complete evidence recovery.

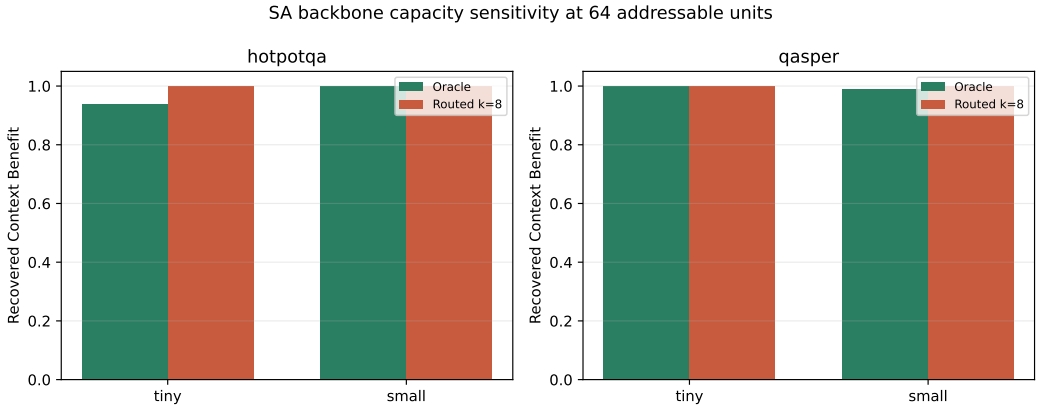

Figure 14: Backbone capacity at 64 units. The small tier improves annotated-oracle robustness and maintains routed quality, but also increases model and K/V width.

The prototype's systems result is mixed. At $k = 8$, transferred K/V is nearly constant with source length: approximately 23–26 KiB per tiny example and 86–94 KiB per small example. Memory-attention time is about 3–4 ms and 6–8 ms. In contrast, Python routing latency grows approximately linearly with candidate count: at 256 units it is 508–511 ms for tiny and 840–1,026 ms for small. The experiment therefore establishes a favorable K/V sparsity curve but not an end-to-end speedup. Batched/vectorized gist search, an external index, and cache reuse are necessary systems work.

These percentages are active attention working-set and potential transfer savings, not yet reductions in total cache storage. The current in-memory prototype retains native K/V for every encoded chunk; if that complete cache remains on the GPU, its resident allocation does not shrink merely because attention gathers eight chunks. Real device-memory savings require an off-device or tiered backing cache, selective transfer, or eviction. With a fully resident cache, the demonstrated savings apply to gathered attention operands, score workspace, and memory-attention computation rather than to the complete stored cache.

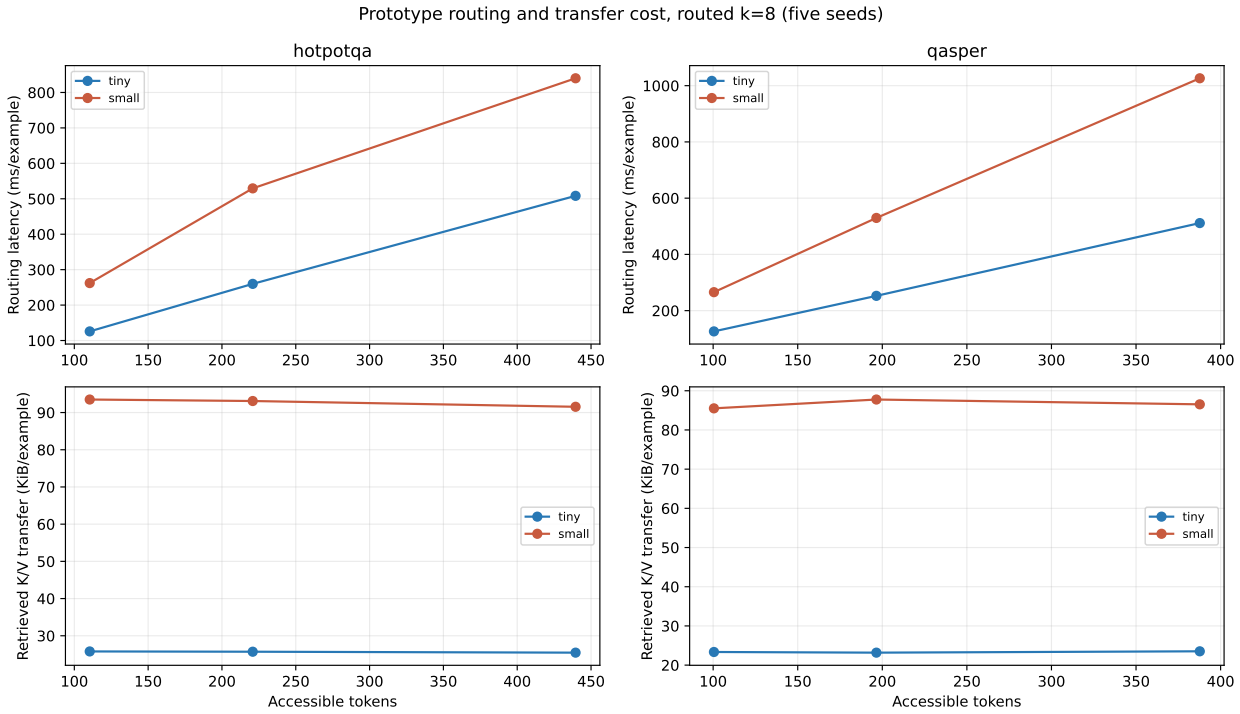

Figure 15: Prototype routing and transfer cost at $k = 8$. K/V transfer remains nearly constant as context grows, but the current Python exhaustive router scales linearly and dominates the attention kernel.

14.4 Exact Tensorized Routing Speed

The quality study exposed a systems question: is exhaustive exact routing intrinsically slow, or was the measured cost caused by candidate-wise Python/CUDA orchestration?

The exhaustive result above isolates a software bottleneck rather than a required PRA operation. The legacy path invokes one small cosine operation per chunk, converts each CUDA result to a Python scalar, and serializes every ranking before applying top-k. The new exact path packs normalized layer gists as $[C, G, D]$, scores the query batch in one matrix operation (a matrix multiply when $G = 1$ and an einsum otherwise), aggregates URI scores on device, and applies `torch.topk` independently to references and chunks. It retains the legacy implementation as a parity control. Cache mutation invalidates the packed index; detached persistent caches can reuse it. Summary-combination and reference-first policies currently retain the scalar fallback.

The optimization changes how the same exact scores are computed and selected; it does not introduce approximate retrieval. The removed work is the Python candidate loop, many tiny CUDA launches, device-to-host scalar synchronization, and full-ranking serialization.

We reran both backends on the same trained checkpoints, examples, native K/V, queries, and $k = 8$ selection policy. The benchmark covers 32 examples at every combination of two datasets, two model tiers, four split counts, and five seeds: 2,560 routed examples per backend. Both kernels are warmed on the same example, backend order alternates by seed, and CUDA is synchronized at phase boundaries only in timing mode. Reported routing time includes per-example packed-index construction but excludes resolver, reference encoding, materialization, and memory attention. Complete-rank serialization is disabled; selected hits and candidate counts remain available.

At 256 splits, tensorized routing takes 27.7–28.0 ms for tiny and 53.2–55.5 ms for small, versus

Table 8: Legacy scalar versus exact tensorized routing latency in ms/example. Values are five-seed means over 32 examples per seed. Speedup is computed within seed before averaging; every paired backend loss is identical to recorded precision.

		32 splits			256 splits		
Dataset	Tier	Legacy	Tensor	Speedup	Legacy	Tensor	Speedup
HotpotQA-derived	tiny	65.0	9.5	6.86×	542.1	28.0	19.38×
HotpotQA-derived	small	142.0	21.7	6.56×	907.9	55.5	16.47×
QASPER-derived	tiny	68.4	10.4	6.63×	522.3	27.7	18.97×
QASPER-derived	small	138.9	20.1	6.95×	980.6	53.2	18.51×

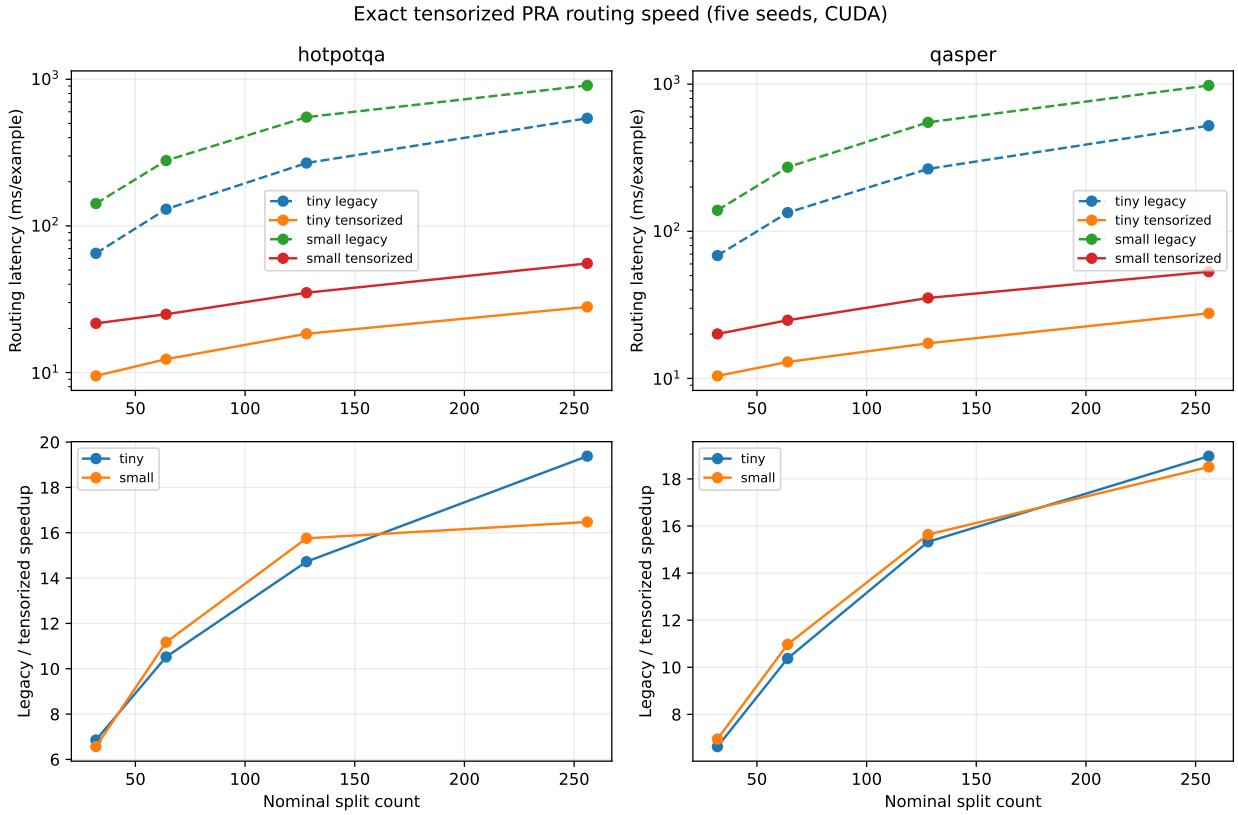

Figure 16: Exact routing latency and paired speedup at 32/64/128/256 nominal splits. The tensorized path still scores every candidate, but replaces hundreds of scalar launches and synchronizations with packed kernels; its advantage therefore grows with addressability.

522–542 ms and 908–981 ms for legacy routing. The speedup grows from 6.56–6.95× at 32 splits to 16.47–19.38× at 256. This removes the dominant prototype artifact identified in Figure 15, but it is not an end-to-end serving result. The benchmark constructs an index for each example; a shared cache can amortize that work, whereas much larger collections may require an approximate index. Conversely, resolver and encoding time, a fully resident K/V cache, Python materialization, and unfused variable-length attention remain outside this routing claim.

14.5 Persistent Index Reuse and Selected-Only Serialization

The next question is how much of exact routing remains when the same source cache serves repeated autoregressive queries. We call index construction plus one query *cold* routing and a query against an already packed index *warm* routing.

The first tensorized benchmark above retained the complete ranking for diagnostics and built the packed index inside every timed request. The serving-oriented path now keeps a detached cache's packed index across queries, gathers only rows belonging to selected references before chunk top-k, and transfers only selected identifiers and scores to the host. These changes preserve exact hierarchical selection; they reduce orchestration, not the candidate set or materialized token K/V.

Table 9: Five-seed exact routing latency in ms/query on real encoded caches. Cold includes index construction; warm reuses it. Every selected URI, chunk, score, and downstream loss matches the scalar implementation.

		32 units			256 units		
Dataset	Tier	Scalar	Cold	Warm	Scalar	Cold	Warm
HotpotQA-derived	tiny	65.20	10.72	7.74	562.90	18.91	5.49
HotpotQA-derived	small	172.34	17.77	11.10	1182.78	29.55	11.56
QASPER-derived	tiny	70.61	10.69	8.53	580.96	14.80	6.28
QASPER-derived	small	172.62	18.59	11.94	1164.30	33.67	11.46

At 256 units, index construction accounts for roughly 45–54% of cold exact routing. Warm routing takes 5.5–6.3 ms for tiny and 11.5–11.6 ms for small, a 94–103× improvement over scalar selection. The tiny index occupies about 260 KiB and the small index about 1.03 MiB. Exact warm search is therefore no longer the dominant operation in this regime; approximate nearest-neighbor search should be justified at larger scales rather than assumed necessary here.

14.6 Long-Prompt Implicit-Head Probe

This experiment asks whether a fact wholly outside a fixed direct window remains usable when the displaced prefix is represented as ordinary PRA memory rather than silently truncated.

For `prompt_overflow_mode=implicit_reference`, the implementation now encodes the displaced head once as causal history, captures native layer K/V, and slices those tensors into routable chunks. It does not reset position or contextual state at each chunk boundary. The direct tail begins at the displaced-token offset, including mixed batch rows with different head lengths. An exact unit test restores every head slice and matches dense full-sequence tail logits.

A five-seed controlled answer-code probe trains a fixed-target model and evaluates total prompt lengths of 24, 48, 96, and 192 tokens against a 24-token direct budget. At 48 and 96 tokens, historical routed loss is 0.929 and 0.939, compared with dense loss 0.929 and 0.944. At 192, routed loss is 1.365 versus 1.498 for dense and 1.772 for truncation while activating 45.8% of source K/V. Target-chunk recall decreases from 1.000 at 48 to 0.894 at 192. Wrong-memory loss is 1.723, and independently

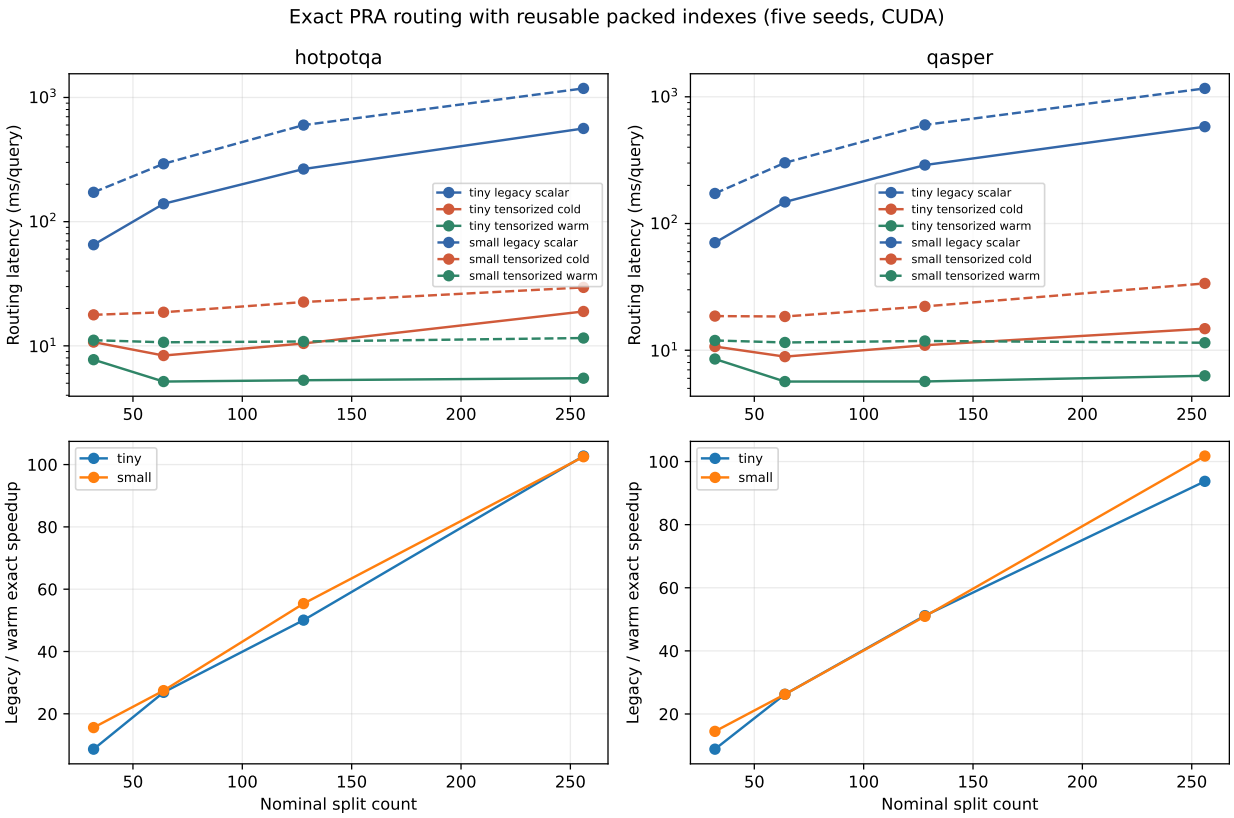

Figure 17: Cold and persistent-index exact routing. Index reuse and selected-only host serialization preserve the full selection semantics while flattening warm latency through 256 addressable units.

encoding head chunks raises loss to 1.929. The result demonstrates causal access beyond the direct window on this controlled distribution; it does not establish unrestricted long-context QA.

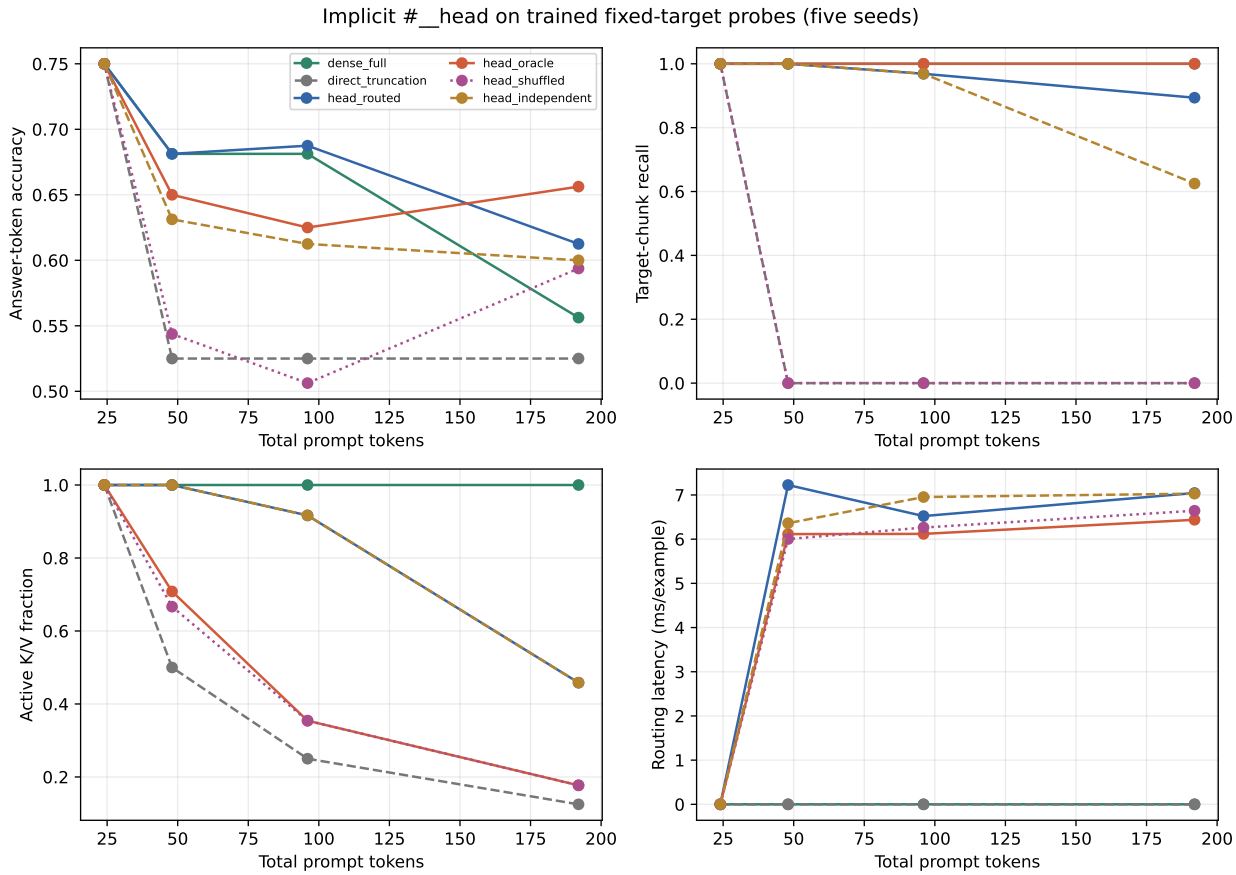

Figure 18: Implicit-head quality through eight times the direct budget. Historical slicing, truncation, wrong memory, and independent chunk encoding isolate access, content causality, and contextual fragmentation.

The 192-token sensitivity sweep exposes a precision–recall tradeoff. Increasing selected chunks from $k = 2$ to $k = 16$ raises target recall from 0.425 to 1.000 but worsens loss from 1.455 to 1.648 because more irrelevant native K/V competes in the shared softmax. Smaller chunks improve loss despite lower target-chunk recall, and a two-token overlap does not help this task. Thus routing recall alone is insufficient: quality must be reported with active K/V and counterfactual loss.

14.7 Model-Bounded Context and Streaming Probe

The preceding probe encoded its whole head in one pass. The model-bounded experiment asks three separate questions: can every base-model operation obey a hard limit, can bounded encoding preserve enough context across block boundaries, and how does an independent materialization budget control recovery?

We next impose a hard 32-token native limit on the same five trained seeds while retaining an 8-token direct tail and a 4-token safety reserve. Sixteen-token encoding cores receive up to four left-context tokens; their stored core K/V is sliced into four-token routing chunks. At the largest condition, 192 logical tokens are 6× the native limit and the displaced 184-token head alone is 5.75×

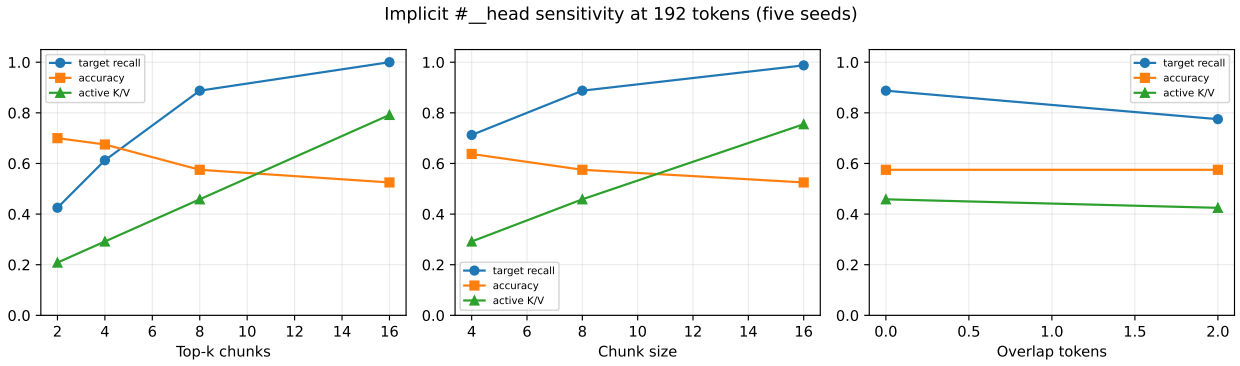

Figure 19: Implicit-head sensitivity at 192 tokens. Greater top-k improves target-chunk recall but can reduce language-model quality by admitting irrelevant K/V.

that limit. This requires 2, 4, 8, and 12 encoding calls as the head grows from 24 to 184 tokens. The largest observed bounded encoding call is 20 tokens—only 62.5% of the 32-token ceiling—and no encoding or attention operation violates the hard limit.

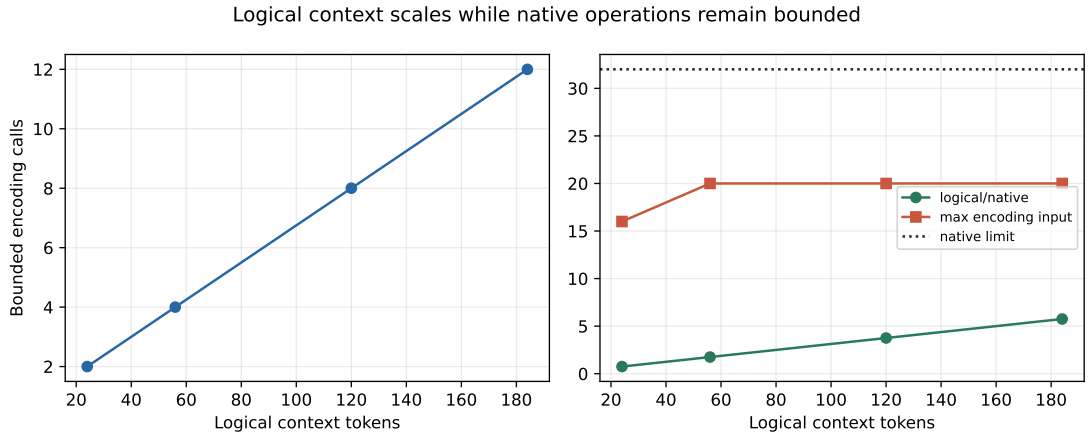

Figure 20: Model-scaled context safety. Logical head length and encoding-call count grow, while each overlap-aware base-model encoding remains below the 32-token hard limit.

At 192 logical prompt tokens, historical overlap routing outperforms direct truncation by 0.172 loss and raises accuracy by 3.75 points. Independent encoding blocks lose this benefit, reaching 1.709 loss. The approximate oracle is better still at 0.874. Shuffled memory is worse than routed memory but remains better than truncation, so this small probe supports useful displaced access without proving that all improvement is content-causal. The dense result is not an upper bound: this model was trained on shorter fixed-target patterns and degrades when every distractor remains active in one softmax.

The observation is that routed loss remains 1.0567 under the hard bound, while independent blocks reach 1.7089. The interpretation is that overlap-aware contextual encoding matters more here than the 32-token operation limit itself. The justified claim is narrower: **logical context can exceed the base model's native context while every underlying operation remains bounded**, provided that routing and materialization recover useful history. Dense access is a control in this regime, not a guaranteed loss oracle.

Table 10: Five-seed bounded-context result at 192 logical tokens; 80 evaluated cases per condition. Loss and accuracy concern the first answer token.

Condition	Loss	Accuracy	Target-chunk recall
Dense full	1.7145	0.5125	1.0000
Direct truncation	1.2287	0.6500	0.0000
Historical routed	1.0567	0.6875	0.5250
Approximate oracle	0.8738	0.7125	1.0000
Shuffled/wrong chunks	1.1205	0.6375	0.0000
Independent encoding blocks	1.7089	0.6375	0.2625

The labels in Table 10 require care. **Dense full is a control, not automatically an oracle**: this tiny model sees a longer sequence and a different distractor/softmax regime than in its fixed-target training distribution. The approximate-oracle condition more directly tests whether known displaced evidence remains useful after bounded encoding and native-K/V transport. Its 0.8738 loss shows that the selected evidence is useful; the routed-oracle gap then measures imperfect selection and admission in this probe. Historical routed loss below dense loss must not be generalized into a claim that sparse attention is intrinsically better than dense attention.

The whole-chunk budget sweep uses eight routing candidates at 192 logical tokens. Every four-token increase admits one additional routing chunk. Recall rises monotonically from 0.325 to 0.525, and loss falls from 1.242 to 1.057. Utilization is exactly 1.0 at every setting because four-token chunks divide all tested budgets. Mean forward latency remains 18.5–19.3 ms and native-memory attention 3.7–4.0 ms on the GTX 950M; these small differences are not a speed trend. Peak allocation rises only from 9.573 to 9.598 MiB. Selected K/V is GPU-resident in this sweep, so measured host-transfer bytes are correctly zero; a CUDA regression separately verifies that budget-rejected CPU chunks are not transferred.

Materialization budget is an independent quality/compute control. Routing can rank the right chunk while admission still rejects it; conversely, increasing the budget may admit useful K/V without changing candidate scores. The four-token safety reserve is withheld from memory admission so generation or task-specific direct tokens can be appended without immediately violating the native limit.

Table 11: Materialization budget at 192 logical tokens. Each row aggregates 80 cases.

Budget	Materialized	Rejected chunks	Recall	Loss
4	4	7	0.3250	1.2416
8	8	6	0.3500	1.2185
12	12	5	0.3750	1.1998
16	16	4	0.4375	1.1741
20	20	3	0.5250	1.0567

Finally, deterministic generation continues for 16, 32, or 48 new tokens. The 48-token condition performs 11 rollovers, migrates 44 tokens, retains a five-token direct suffix, and routes memory on 40 generation steps. Its maximum per-layer materialization is 20 tokens and maximum native operation is 20, below the configured limit of 32 in all five seeds. This is a mechanism smoke test, not a generation-quality result; rebuilding the entire head after each rollover remains the dominant avoidable cost.

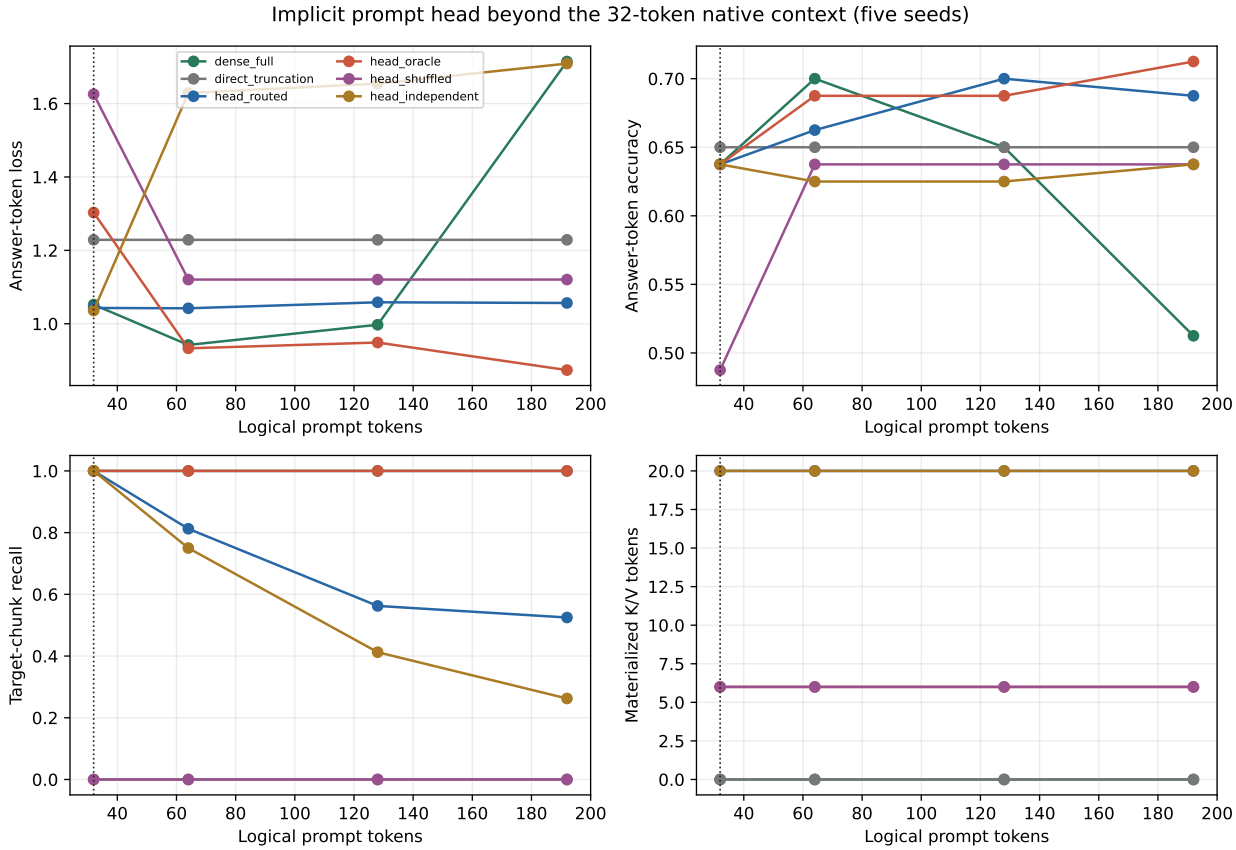

Figure 21: Quality, recall, and active memory as the implicit head grows beyond the native limit. Dense execution is shown only because this tiny source model supports 192 tokens.

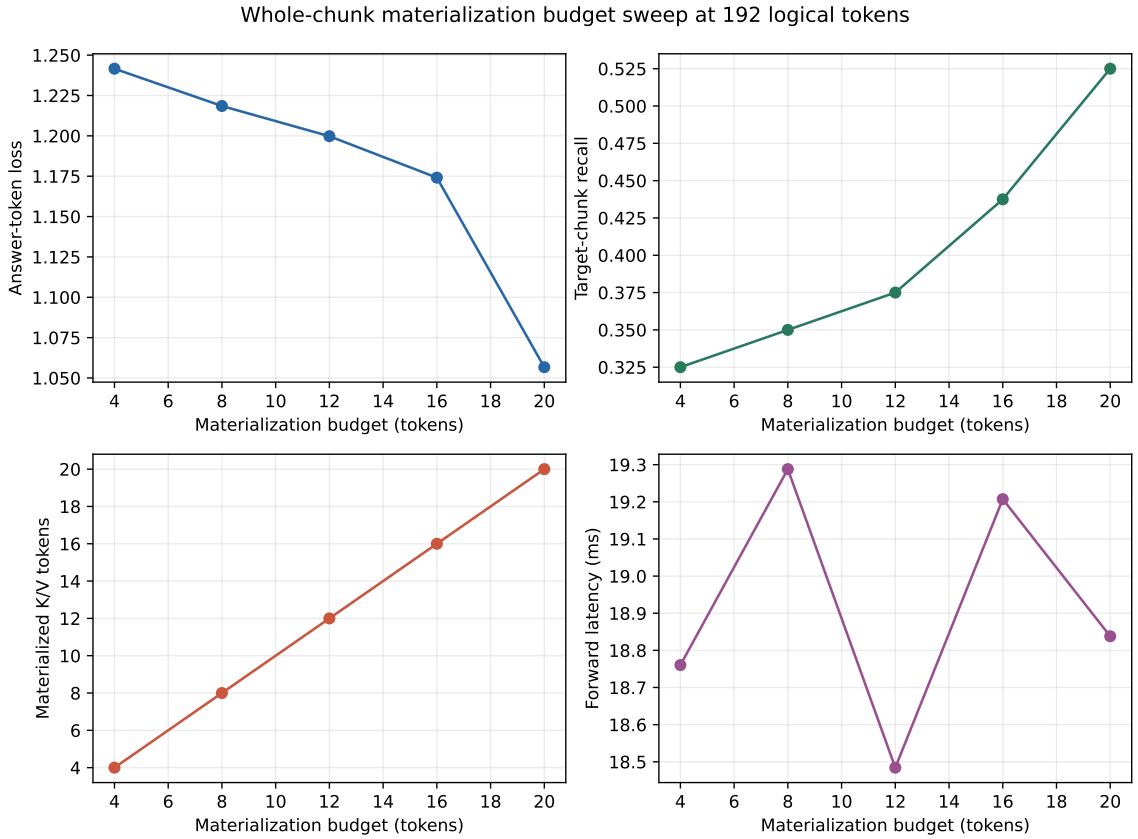

Figure 22: Quality and systems measurements under score-ordered whole-chunk admission. The sweep changes active K/V, not routing candidate count.

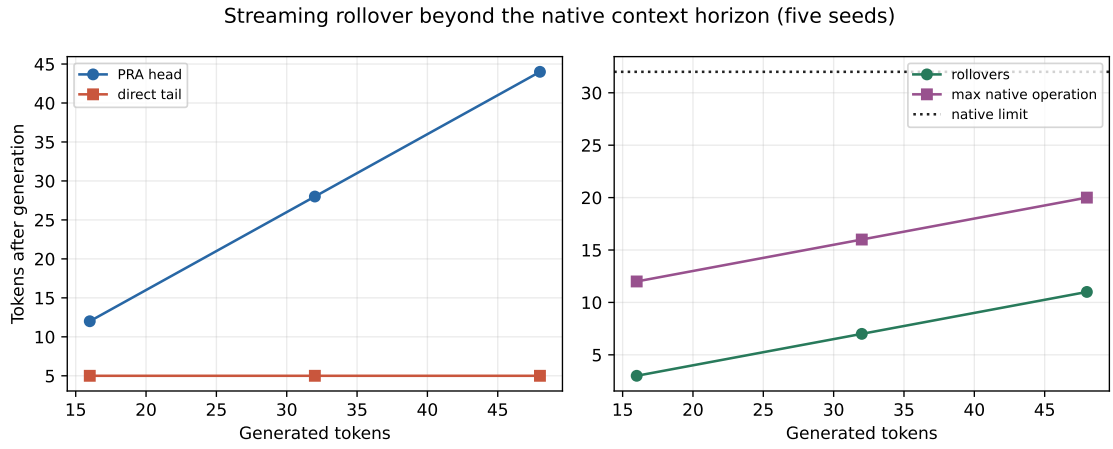

Figure 23: Streaming rollover beyond the native horizon. Direct history remains bounded; expired tokens accumulate in the routable request-local head without a native-limit violation.

14.8 CPU-Resident Native K/V Prototype

Active K/V sparsity reduces attention work, but it does not reduce GPU cache residency when all source K/V remains on device. This experiment asks whether selected-only transfer can turn sparse activity into measured GPU-capacity reduction without changing selection or loss.

The prototype can now retain routing gists and their packed index on GPU while placing complete detached token K/V in pinned CPU memory. Materialization transfers only selected chunks. Across five seeds, two datasets, two model tiers, and 32–256 units, CPU and GPU residency produce identical selected chunks and zero measured loss delta.

Table 12: CPU residency at 256 units. Removed K/V is complete source payload no longer resident on GPU; selected transfer and latency are warm-request means.

Dataset	Tier	Removed MiB	Transfer KiB	Transfer ms	Warm GPU/CPU ms
HotpotQA-derived	tiny	0.855	25.4	4.7	18.52 / 22.54
HotpotQA-derived	small	3.420	87.9	8.9	35.63 / 45.87
QASPER-derived	tiny	0.734	24.6	4.0	18.20 / 22.64
QASPER-derived	small	2.936	85.7	8.4	32.67 / 46.19

Measured peak allocation falls by approximately the removed payload: 0.70–0.84 MiB for tiny and 2.80–3.34 MiB for small. Cold construction is 3–4× slower because the prototype issues many small synchronous offloads. This validates selective residency and accounts for its cost; batched offload, asynchronous overlap, and a fused variable-length attention kernel remain necessary systems work.

15 Historical Version-1 Results

Every result from this section through the corrected five-seed reference adaptation study uses the optional cross-attention transport. The values remain informative about adaptation and preservation, but they are not evidence for native-KV equivalence, sparsity, or inference economics.

Tables 13 and 14 report population means and standard deviations over seeds 1 and 7. They are real report artifacts, but the reference runs predate the corrected hierarchy. They routed a single raw summary embedding per whole reference, reused a batch-collapsed query in an early path, and materialized whole-reference K/V. Accordingly, only the no-memory language-model results transfer directly to the current implementation; reference-conditioned values are engineering history, not evidence for corrected PRA.

Table 13: Phase-1 WikiText-2 language modeling at a matched 524,288-token budget.

Architecture	Parameters	Test loss	Perplexity	Token acc.	Train s
SelfAttention	4,216,320	$4.7691 \pm .0036$	$117.82 \pm .42$	$.1264 \pm .0006$	94.07 ± 1.43
Full PRA	4,479,488	$4.7457 \pm .0011$	$115.09 \pm .13$	$.1250 \pm .0012$	90.86 ± 1.13
Hybrid PRA	4,347,904	$4.7574 \pm .0017$	$116.44 \pm .19$	$.1271 \pm .0004$	$95.66 \pm .04$

All three architectures learn an ordinary token-prediction function and finish within 0.024 test-loss units of one another. This supports the narrow claim that a PRA block behaves comparably to the local baseline when no external memory is present at this scale and budget. It does not establish equivalence under scaling, and the models are not parameter matched.

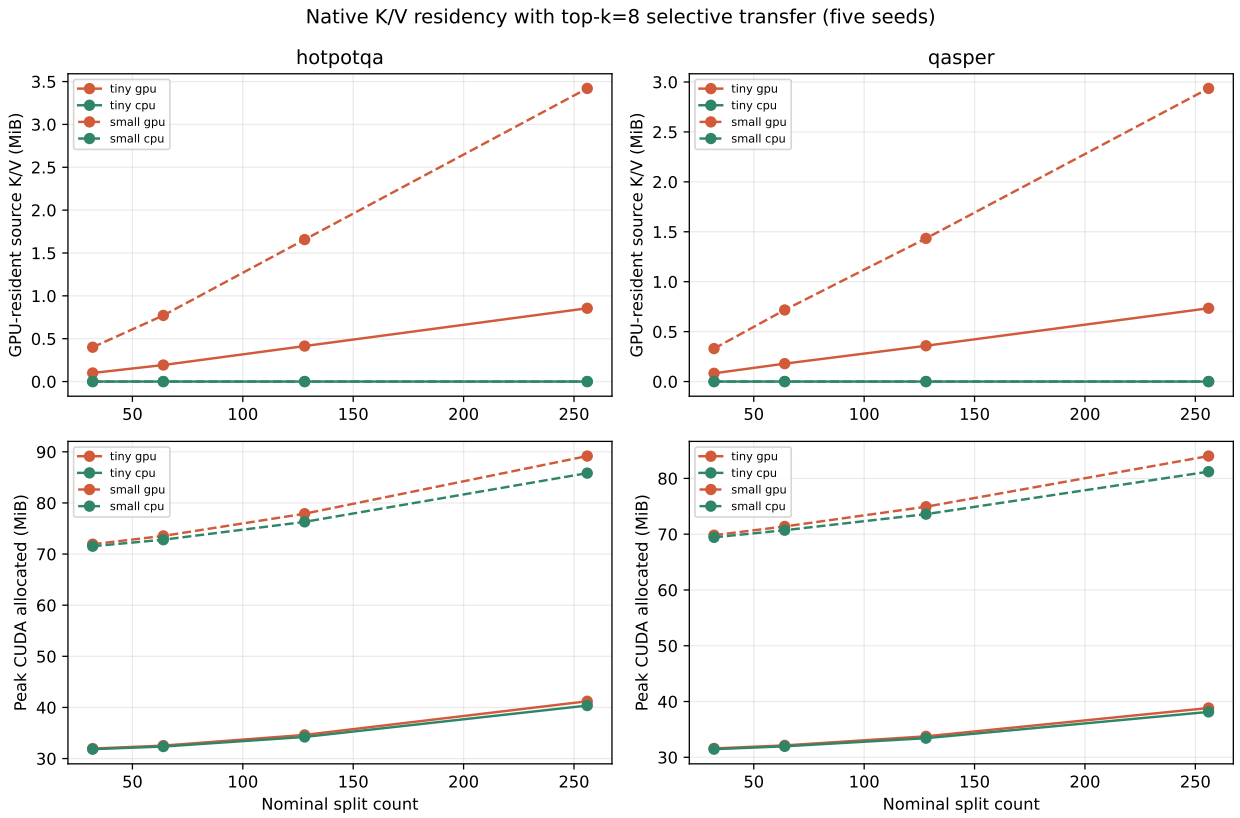

Figure 24: GPU capacity and warm latency under GPU- versus CPU-resident source K/V. Fixed top-k bounds selected transfer, but host backing introduces a measurable latency cost.

Table 14: Phase-2 reference-conditioned answer-token loss and plain-text retention. Lower is better.

Architecture	Valid	Disabled	Shuffled	Plain Δ	Train s
SelfAttention	$4.7791 \pm .0734$	$4.7791 \pm .0734$	$4.7791 \pm .0734$	+0.1859	$44.82 \pm .45$
Full PRA	$4.6749 \pm .0607$	$4.7914 \pm .0683$	$4.6894 \pm .0563$	0.0000	45.13 ± 2.83
Hybrid PRA	$4.7111 \pm .0356$	$4.7953 \pm .0284$	$4.7233 \pm .0351$	0.0000	$31.14 \pm .02$

In that implementation, disabling reference memory increased mean loss by 0.1165 for full PRA and 0.0841 for hybrid PRA, while shuffling increased loss by 0.0145 and 0.0122. These deltas show that the old residual branch changed predictions, but cannot establish correct per-example chunk selection under the corrected semantics. They motivate, but do not substitute for, rerunning counterfactuals with the new reference/chunk traces.

Plain-text reevaluation after phase 2 exactly reproduces the phase-1 losses for full and hybrid PRA because frozen base parameters and zero-initialized isolated output projections protect the local path. The SelfAttention control's mean plain loss rises from 4.7691 to 4.9550, a 0.1859 increase corresponding to about a 20% perplexity increase. This demonstrates preservation by the chosen training strategy, not an inherent guarantee of PRA: the control updates all parameters while PRA updates only reference-specific projections.

Phase-1 mean synchronized validation times are 1.07–1.14 seconds and mean wall-clock times are 97.0–102.0 seconds. Phase-2 mean synchronized training times are 31.1–45.1 seconds, with 46,301.5 mean processed prompt/answer tokens and 512 optimizer steps. These historical timings predate the batch-aware prompt-forward path and must not be used as measurements of its speedup. New reports should separate reference-cache construction, row-local routing, the single prompt forward, and bucketed memory attention.

16 Corrected Five-Seed Controlled Follow-Up

The corrected study executes 65 CUDA runs: 25 ordinary-language runs (five architecture/capacity groups times five seeds) and 40 fixed-reference runs. Every condition uses paired model seeds $\{1, 7, 21, 42, 87\}$, the same fixed tokenizer and data split, and the corrected per-row, per-layer chunk router. Tables report means and population standard deviations over seeds. With five pairs, a two-sided exact sign-flip test cannot attain $p < 0.05$ when every paired difference has the same sign; its minimum is $p = 0.0625$. We therefore emphasize paired effect direction and magnitude rather than binary significance.

Table 15: Five-seed plain WikiText-2 follow-up at a 1,048,576-token budget. Active time is synchronized GPU training time.

Architecture	Parameters	Test loss	Train tok/s	Active s
SelfAttention, same width	4,216,320	$4.5704 \pm .0076$	5411 ± 125	193.9 ± 4.5
Full PRA	4,479,488	$4.5034 \pm .0163$	5623 ± 110	186.4 ± 3.4
Hybrid PRA	4,347,904	$4.5500 \pm .0112$	5438 ± 123	192.9 ± 4.4
SelfAttention, matched to full	4,478,976	$4.5855 \pm .0347$	–	–
SelfAttention, matched to hybrid	4,347,648	$4.5715 \pm .0082$	5657 ± 38	185.4 ± 1.2

Full PRA is 0.0820 loss units better than its nearly parameter-matched SelfAttention control, and hybrid PRA is 0.0215 better than its matched control; every paired seed has the same sign. The

exact paired p-value is nevertheless 0.0625 in both comparisons. These results support ordinary-language compatibility and motivate replication; they do not establish architectural superiority or statistical equivalence. Relative to the historical 524,288-token pilot, full-PRA mean test loss falls from 4.7457 to 4.5034, a 0.2423 reduction. The longer tested pretraining schedule helps, but it does not identify an optimum. One matched-full run resumed after a host-side wrapper timeout, so its quality result is retained while its aggregate timing is omitted from the table.

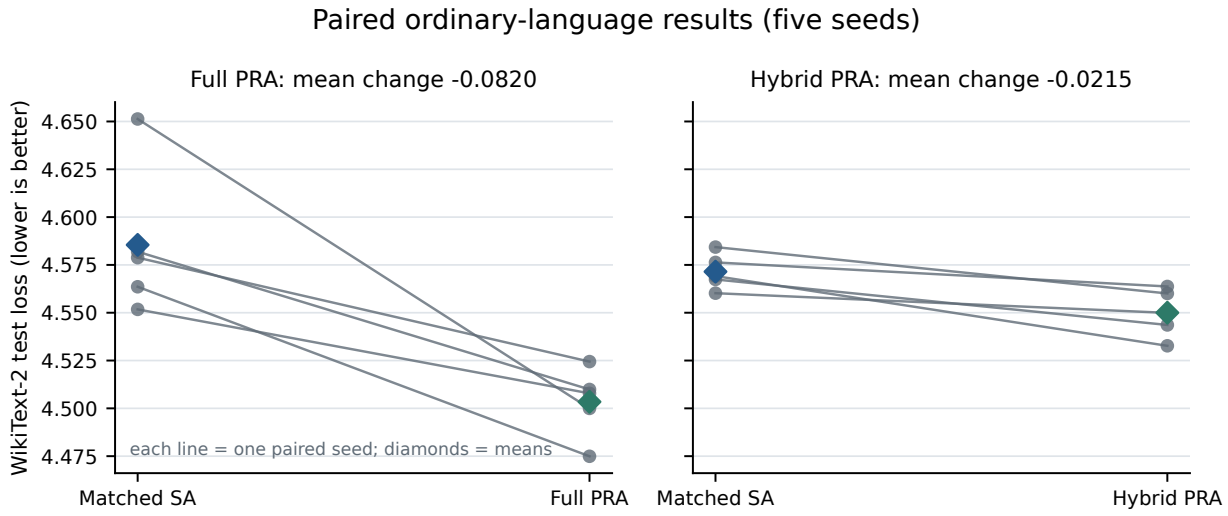

Figure 25: Paired ordinary-language losses for the historical cross-attention-capable backbones. Each line connects one seed to its nearly parameter-matched SelfAttention control; diamonds mark five-seed means. External memory is absent in these runs. The directional result is encouraging but underpowered ($p = 0.0625$), and it is not a native-KV quality measurement.

Table 16: Corrected-router reference-task training order. Values are five-seed means; lower loss is better.

Architecture	Order	Valid	Disabled	Plain after	Top-1	Train tok/s
SelfAttention	scratch	6.3459	6.3459	6.4787	–	1615
SelfAttention	joint	4.9230	4.9230	4.7125	–	–
Full PRA	scratch	6.3557	6.3754	6.4948	.4212	983
Full PRA	frozen path	4.6720	4.6895	4.5034	.4221	1244
Full PRA	joint	4.9129	4.8938	4.6809	.4269	1052
Hybrid PRA	scratch	6.3642	6.3846	6.5140	.4231	1157
Hybrid PRA	frozen path	4.7296	4.7413	4.5500	.3981	1622
Hybrid PRA	joint	4.9196	4.9125	4.7062	.4077	1141

Pretraining strongly improves final reference-task loss. Frozen-path training improves over scratch by 1.6837 for full PRA and 1.6346 for hybrid PRA; direct joint training improves by 1.4429 and 1.4447. Each paired effect has the same sign in all five seeds. Frozen training exactly reproduces the corresponding 4.5034 and 4.5500 plain losses because the base is frozen. Direct joint tuning instead increases plain loss by 0.1775 for full PRA and 0.1562 for hybrid PRA. It also finishes 0.2408 and 0.1899 loss units worse than frozen-path adaptation. Direct joint fine-tuning is therefore not sufficient under this budget. The hybrid does not benefit more from staging than full PRA: its

scratch-to-frozen improvement is smaller.

Table 17: Counterfactual losses after corrected frozen-path training. Five-seed means.

Architecture	Valid	Disabled	Shuffled	Irrelevant	Oracle
Full PRA	4.6720	4.6895	4.6769	4.6723	4.6702
Hybrid PRA	4.7296	4.7413	4.7304	4.7304	4.7279

The frozen memory branch contributes relative to disabling it, by 0.0175 loss for full PRA and 0.0117 for hybrid PRA. Correct content is not yet the source of that advantage: shuffled-reference penalties are only 0.0048 and 0.0008, irrelevant penalties are below 0.001, and oracle references improve loss by only 0.0018 and 0.0017. Full-PRA test top-1 is 0.4221 ± 0.0156 and hybrid top-1 is 0.3981 ± 0.0397, against a candidate-count-weighted chance rate of 0.4202. Periodic curves remain flat rather than showing faster selection after pretraining. Thus pretraining improves language competence and adaptation loss, while the current unsupervised discrete router does not learn generalized reference selection.

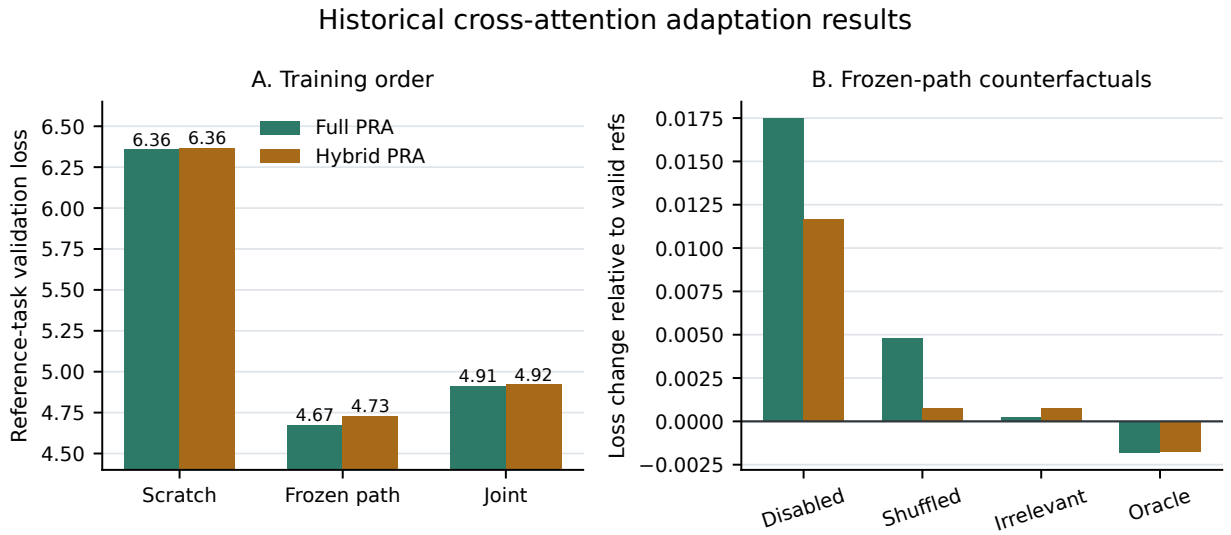

Figure 26: Historical cross-attention adaptation results. Left: frozen-path adaptation finishes substantially below scratch and direct joint tuning for both PRA placements. Right: frozen-path loss changes relative to valid references; positive values are worse. Disabling memory has a visible cost, but shuffled and irrelevant changes and oracle gains are small, so the branch effect is not evidence of content-causal retrieval.

The timing counters also separate optimization conclusions from systems claims. Across uninterrupted runs, full-PRA scratch, frozen, and joint training process approximately 983, 1,244, and 1,052 tokens/s; hybrid variants process 1,157, 1,622, and 1,141 tokens/s. Frozen training is faster because only the memory output path receives gradients. These are single-device prototype measurements with per-example reference encoding and no persistent cache reuse. They characterize this implementation, not an efficiency advantage over long-context baselines.

17 Post-Refactor Fixed-Split Smoke Study

After separating the generic training engine into `src/common`, we reran the standalone synthetic workflows, plain WikiText-2 smoke notebooks, and a corrected-router fixed-split matrix. The matrix uses 32 WikiText-2 source entries, paired model seeds {1, 7, 21, 42, 87}, a fixed data-generation and split seed of 1729, one shared 1,331-token BPE vocabulary, four layers, width 256, context length 128, batch size 2, and eight optimizer steps. Split count means total text parts including the prediction tail: split 2 exposes one earlier part, while split 5 exposes four. Source entries, tokenizer, optimizer-step budget, and sequence length are shared across the 20 runs. The target span and processed-token count nevertheless change with the partition, so the comparison is a pipeline and scaling diagnostic rather than an isolated estimate of split-count causality.

Table 18: Post-refactor CUDA smoke matrix. Values are population mean ± standard deviation over five paired model seeds. $\Delta_{\text{use}} = L_{\text{disabled}} - L_{\text{valid}}$ and $\Delta_{\text{content}} = L_{\text{shuffled}} - L_{\text{valid}}$.

Architecture	Splits	Tokens	Test loss	Δ_{use}	Δ_{content}	Train s
SelfAttention	2	903.0	7.1844 ± 0.0350	0	0	0.581 ± 0.068
SelfAttention	5	1,168.4	7.0881 ± 0.0646	0	0	0.884 ± 0.085
Full PRA	2	903.0	7.2207 ± 0.0310	0.0189 ± 0.0030	0.0002 ± 0.0006	0.959 ± 0.218
Full PRA	5	1,168.4	7.0357 ± 0.0542	0.0312 ± 0.0048	-0.0006 ± 0.0009	1.443 ± 0.224

The SelfAttention invariance across valid, disabled, and shuffled conditions is a useful control. Every PRA seed has a positive disabled-path gap in both partition regimes, so memory-branch activation is reproducible rather than a single initialization artifact. Correct content is not. Mean shuffled-reference changes are effectively zero, and split-5 reference top-1 is 0.150 ± 0.102, below the 0.25 uniform-candidate rate. Split-2 top-1 is trivially 1.0 because only one candidate exists. Split 5 raises mean synchronized training time by about 52% for both architectures while increasing processed target-bearing tokens by about 29%. PRA is about 63–65% slower than SelfAttention in these runs. Because each run lasts less than two seconds, startup, synchronization, unequal parameter counts, and unequal token totals preclude quality or systems superiority claims. The defensible contribution is narrower: the corrected batched path, counterfactual evaluator, paired-seed aggregation, split metadata, timing counters, notebook outputs, and HTML/JSON reports execute together without regression.

17.1 Reproducibility and Artifact Scope

Resolved per-seed HTML and JSON reports, plots, timing counters, checkpoints, and aggregate reports are generated under `out/reports`; the consolidated study is in `out/reports/phase1_followup_v2_5seed_findings`. The aggregate computes population mean and standard deviation without discarding individual seed values and records the fixed data-generation and split seed. Active GPU training time is reconstructed from synchronized batch durations. Content hashes, peak allocated memory, matched token and trainable-capacity budgets, longer training, and independent replication remain necessary before treating the result as publication-grade.

18 Relationship to Prior Work

PRA combines mechanisms that also appear in several mature research lines. The comparison below is organized by what each line makes sparse or persistent; no single component is claimed as

unique.

Sparse attention. Longformer and BigBird impose local/global sparsity, Reformer hashes token queries, and Routing Transformer clusters content within a sequence [2, 20, 8, 16]. These methods primarily change the interaction pattern among tokens already admitted to a long sequence. PRA instead keeps candidate context in an address space and materializes selected historical K/V, so admission and attention sparsity are separately measurable.

KV selection and compression. H_2O retains heavy-hitter cache entries, while SnapKV and ClusterKV select or merge prompt K/V to reduce decoding-time cache cost [21, 11, 12]. PRA shares the goal of reducing active K/V, but its default selection unit is a provenance-bearing chunk named by a reference, and selected chunks retain their full native token K/V. These approaches are complementary: PRA can choose objects or chunks before a token-level eviction or compression policy acts.

External and retrieved memory. Memorizing Transformers retrieve previous activations, RETRO conditions on database chunks, and RAG/REALM retrieve non-parametric evidence [17, 3, 10, 6]. PRA differs in the runtime contract: explicit local handles bind URIs, references may form a bounded graph, and the canonical transport reuses layer-native K/V rather than appending retrieved text or using a separately trained cross-attention encoder.

Recurrent and long-term memory. Transformer-XL recurs over segment states, Compressive Transformer compresses older activations, Infini-attention combines local attention with compressive memory, and TransformerFAM and Titans introduce persistent or learned long-term memory [4, 15, 14, 7, 1]. PRA's implicit head is related in purpose but remains an addressable, selectively materialized history. Its current bounded-block encoding is an approximation rather than a claim of exact recurrence beyond the base model's native horizon.

Landmarks and routing summaries. Landmark Attention uses landmark tokens to organize random access to blocks, and StreamingLLM identifies stable attention sinks for streaming windows [13, 18]. PRA gists likewise support coarse selection, but they are an index over URI/chunk records and are not normally transported as the final memory representation. The present router is exact over its packed candidate set; approximate search remains an optional systems choice at larger scales.

Serving systems. vLLM's PagedAttention demonstrates the importance of paging and sharing K/V during serving [9]. PRA adds a semantic selection layer but inherits the same need for admission, batching, eviction, cache identity, and isolation. The CPU-residency experiment is therefore a prototype accounting result, not a substitute for a production pager.

19 Safety and Systems Correctness

Reference memory expands the model's attack and failure surface. A malicious or compromised reference can poison summaries, embed adversarial instructions, or exploit trusted URI namespaces. URI resolution must therefore preserve source, content digest, version, authorization decision, and resolver identity. Entries from different trust domains should not be merged into an anonymous K/V pool, and outputs should retain enough provenance to reconstruct which objects were resident.

Freshness is part of correctness. A cache key based only on URI can return stale memory after content or model parameters change. Persistent implementations should bind entries to content version, model/adapter identity, layer, representation format, and policy domain. Revocation must invalidate both object lookup and already encoded resident representations.

Batching creates a confidentiality boundary. The corrected prototype constructs a cache per example after a batch-union design exposed cross-example references, then places those caches behind a row-indexed wrapper for one prompt forward. Optimized kernels must preserve equivalent masks and should include adversarial tests proving that logits for one sequence are invariant to another sequence's private cache. Cache reuse across users or tenants requires authorization-aware deduplication.

Recursive expansion can amplify work. The standalone builder now bounds depth, child count, total references, and encoded tokens; detects cycles; exposes missing-reference and overflow policies; and prevents partial entries from becoming searchable. It also fingerprints content, model, tokenizer, and routing configuration. Authorization, latency/byte budgets, revocation, cryptographic provenance, and durable cache invalidation remain deployment requirements beyond the in-memory resolver.

20 Limitations

Task and scale. The HotpotQA- and QASPER-derived tasks inject a fixed answer code and therefore test transport over natural text, not unrestricted multi-hop or scientific-document QA. One contextualized answer URI can be sufficient even when annotated evidence coverage is incomplete, so near-perfect loss does not establish complete semantic retrieval. The backbones are small and trained from scratch. The historical corrected-router smoke matrix uses only 32 source entries and eight optimizer steps, and its weak content counterfactuals do not establish learned reference use. No foundation-model or broad superiority claim is made.

Model integration and selection. The backbone uses learned absolute positions; relative positional schemes such as RoPE may change the positional component of forced block fragmentation and stored-K portability. A separate controlled Paper 1.5 study is deferred before pretrained-model integration. Historical slicing has not been validated with grouped-query attention or production cache layouts. The selector is heuristic, uses the last valid projected query, receives no auxiliary relevance loss, and applies a nondifferentiable hard top-k. Multi-gist modes have a staged screen but no learned-router comparison. Recursive construction is bounded and deterministic rather than a learned iterative policy.

Runtime. Packed exact routing is measured only through 256 units. Cache construction, selected-record assembly, summary and reference-first routing, and materialization still contain row-wise Python work; variable-length attention is unfused. CPU residency issues many small synchronous offloads, making cold construction substantially slower. Streaming rollover is implemented and preserves the native bound, but it rebuilds the complete implicit-head cache and packed index after each rollover rather than appending K/V incrementally.

Evidence boundary. The controlled studies establish implementation invariants, sparse native transport, fragmentation diagnosis, exact routing parity, and bounded execution. They do not yet show semantic selection on unrestricted tasks, recursive benefit, end-to-end serving speedup, or a favorable comparison with strong long-context, cache-compression, and RAG baselines.

21 Conclusion

The standalone PyTorch PRA prototype now makes native K/V the default transport, converts trained SelfAttention weights without adding transport parameters, and preserves the former cross-attention path for reproducibility. Exact whole-model tests cover historical source K/V and continued tail positions. Native historical slicing preserves dense transport through 256 units, and controlled implicit-head studies recover causally relevant tokens outside the direct window. The model-bounded path now separates logical, native, encoding, and routing scales; enforces one whole-chunk budget across all memory sources; and rolls generated history into routable memory. At fixed $k = 8$, active K/V falls to roughly 5%. Reusable exact indexes reduce 256-way warm routing to 5.5–11.6 ms, while CPU-resident source K/V demonstrates a 0.73–3.42 MiB GPU-capacity reduction at a 4–9 ms warm transfer cost. This establishes mechanisms and concrete prototype tradeoffs, not unrestricted QA or production efficiency. The immediate priorities are genuine QA with a capable pretrained backbone, incremental streaming-cache append, batched asynchronous cache movement, fused attention, and end-to-end comparisons.

A Line-Numbered Reimplementation Guide

This appendix is a compact implementation specification rather than a copy of the repository. Its numbered listings preserve the invariants needed to reproduce native-K/V PRA in another decoder: cache entries are layer specific; routing gists and detailed token K/V are different objects; every batch row owns its cache; historical positions are not reset at URI boundaries; and selected memory joins local K/V before a single softmax. The notation is PyTorch-like and omits error messages, fingerprints, tracing fields, and legacy cross-attention.

A.1 Minimal End-to-End Algorithm

For one request row and decoder layer, a minimal implementation performs the following steps. The same row-private cache is visible at every layer, but its K/V and routing gists are layer specific.

1. Keep the recent prompt suffix as direct context and move overflow token IDs into the request-local `#__head` object.
2. Resolve explicit handles through the current object's local reference table.
3. Partition every long source into encoding blocks no larger than $L_{\text{model,max}}$, optionally adding left context that is not stored.
4. Run the ordinary decoder over each block and capture every layer's native K/V.
5. Discard context-only overlap and slice each encoding core into smaller routing chunks.
6. Pool one or more key gists per chunk and build a packed, normalized layer index.
7. Score the live query against all gists and apply exact hierarchical reference/chunk top-k.
8. Merge candidates from implicit history and explicit references, then admit complete chunks in score order under the single materialization budget.
9. Gather or transfer only admitted K/V; concatenate it before direct K/V.

10. Apply one masked attention softmax and the decoder's existing output projection.

11. Record content, routing, active-K/V, transfer, residency, and timing diagnostics.

12. During generation, move expired routing-sized prefixes from the direct suffix into `#__head`, invalidate its index, and rebuild or incrementally append its cache.

A.2 Implementation Map at Source Revision `5b1d992`

Table 19 anchors the pseudocode to the code revision audited immediately before this documentation-only refinement. Function and class identifiers are the stable anchors; supplemental line ranges were verified against that revision and should be treated as navigation aids rather than API contracts. No implementation file changed in this pass. Each result artifact separately records the earlier `git_sha` that produced it.

Table 19: Artifact-to-source map for independent reimplementation and replication.

Moving part	Source symbol	Lines
A. source preparation	`prompt.py::prepare_prompt_batch_for_pra`	125–242
A. source partitioning	`chunking.py::partition_source`	315–434
B–C. bounded encode/slice	`model.py::_bounded_reference_payloads`	484–630
C. cache publication	`model.py::encode_reference_tokens_to_cache`	838–1091
D–E. gists/index	`memory.py::_tensorized_chunk_index`	457–539
F. exact scoring	`memory.py::_score_chunks_tensorized`	583–640
F. hierarchical top-k	`memory.py::_hierarchical_tensorized`	641–762
G. budgeted selection	`attention.py::_budget_selection`	155–224
H. K/V materialization	`attention.py::_materialize`	225–330
I. one-softmax attention	`memory_batching.py::native_kv_attention`	216–307
J. streaming rollover	`model.py::generate`	1092–1311

The principal experiment entry points, with supplemental source lines at revision `5b1d992`, are:

- `scripts/run_native_kv_benchmark.py::run` (1244), `run_pra_scale_sensitivity.py::run` (783), and `run_pra_parameter_sensitivity.py::run` (552);
- `run_pra_routing_speed.py::run` (251), `run_pra_routing_index_reuse.py::run` (348), and `run_pra_kv_residency.py::run` (188);
- `run_pra_bounded_context.py::run` (340).

Their JSON manifests record seeds, checkpoints, conditions, and output figures.

Table 20: Reproducibility map from reported result families to committed artifacts.

Result family	Machine-readable artifact in docs/papers/shared/results	Runner
Native quality	native_kv_synthetic.json, native_kv_hotpotqa.json, native_kv_qasper.json	run_native_kv_benchmark.py
Parameter/scale	pra_parameter_sensitivity_topk.json, ..._gist.json, ..._fragmentation.json, pra_scale_sensitivity.json	run_pra_parameter_sensitivity.py, run_pra_scale_sensitivity.py
Exact routing	pra_routing_speed.json, pra_routing_index_reuse.json	run_pra_routing_speed.py, run_pra_routing_index_reuse.py
Implicit head	pra_long_prompt_head.json, pra_long_prompt_head_sensitivity.json	run_pra_long_prompt_head.py, run_pra_long_prompt_sensitivity.py
Bounded context	pra_head_beyond_native_context.json, pra_context_budget.json, pra_materialization_budget.json, pra_streaming_rollover.json	run_pra_bounded_context.py
K/V residency	pra_kv_residency.json	run_pra_kv_residency.py

A.3 A. Source Preparation: Cache Records and Resolution

The cache has two levels of identity. A reference entry owns a stable URI and one or more chunks. Each chunk then owns a separate representation at every PRA layer. The token K/V payload is what attention eventually consumes; the much smaller gist set is what routing searches.

```
001 | @dataclass
002 | class LayerKV:
003 |     k: Tensor  # [1, H, M, Dh]
004 |     v: Tensor  # [1, H, M, Dh]
005 |
006 | @dataclass
007 | class RoutingGists:
008 |     k: Tensor  # [G, D], where D = H * Dh
009 |     v: Tensor | None  # [G, D], used only by gist-only materialization
010 |
011 | @dataclass
012 | class ChunkMemory:
013 |     chunk_id: str
014 |     source_uri: str
015 |     token_start: int
016 |     token_end: int
017 |     token_kv: LayerKV
018 |     routing_gists: RoutingGists
019 |
020 | @dataclass
021 | class CacheEntry:
022 |     uri: str
023 |     layer_chunks: dict[int, list[ChunkMemory]]
024 |     state: Literal["BUILDING", "READY", "FAILED"]
```

Here H is head count, D_h head width, $D = HD_h$ model width, M detailed tokens in one chunk, and G gists for that chunk. Line 3 deliberately retains the same projected head layout as ordinary decoder K/V. Line 8 merges heads only for cosine routing; it does not replace line 3. An entry

becomes searchable only in `READY` state, preventing recursive construction from exposing partial layer memory.

A.3.1 Resolution, Recursion, and Publication

Resolution converts a handle into text plus an explicit local reference table. The same budget object follows the recursive traversal, and the current URI path detects cycles. Children are completed before a parent is published, so a parent encoding may route only to complete descendants.

```
025 | def ensure_cached(uri, cache, budget, path):
026 |     if uri in path: return apply_cycle_policy(uri, path)
027 |     if cache.state(uri) == "READY": return
028 |     budget.consume_reference(uri)
029 |     cache.mark_building(uri)
030 |     resolved = resolver.resolve(uri)
031 |     for child in resolved.local_reference_table.values():
032 |         ensure_cached(child.uri, cache, budget, path + [uri])
033 |     entry = encode_reference(uri, resolved.token_ids)
034 |     cache.put_ready(entry)  # atomic visibility boundary
```

Line 31 is intentionally restricted to the resolved object's local table; a global token registry would create accidental edges. Production cache identity must extend the URI with content version, tokenizer, model/adapter, layer format, and policy domain. Otherwise line 27 can return stale or unauthorized K/V.

A.4 B–C. Bounded Encoding and Native-K/V Slicing

Independent encoding resets context and positions for every URI. That is useful as an ablation but is not equivalent to causal history. When the complete source fits, `native_slice` performs one exact pass. The general path below instead bounds each base-model call and slices every encoding core into smaller routing units.

```
035 | def encode_bounded(uri, ids, encode_cfg, route_cfg, Lmax):
036 |     cores = partition(ids, encode_cfg.without_overlap())
037 |     out = []
038 |     for block_id, core in enumerate(cores):
039 |         h = min(core.start, encode_cfg.left_overlap,
040 |                 Lmax - len(core.ids))
041 |         block = ids[core.start-h:core.end]
042 |         assert len(block) <= Lmax
043 |         captured = model.capture_native_kv(block) # layer->[1,H,E,Dh]
044 |         core_kv = {l: (k[:,:,h:h+len(core.ids),:],
045 |                        v[:,:,h:h+len(core.ids),:])
046 |                    for l,(k,v) in captured.items()}
047 |         for route in partition(core.ids, route_cfg):
048 |             a, b = route.start, route.end
049 |             logical = (core.start+a, core.start+b)
050 |             native = {l: (k[:,:,a:b,:], v[:,:,a:b,:])
051 |                       for l,(k,v) in core_kv.items()}
```

```
052 |             out.append(make_chunk(uri, logical, native,
053 |                                   encoding_block=block_id))
054 |     return out
```

Line 41 may contain context that is absent from stored K/V: lines 44–45 discard overlap before lines 47–53 expose smaller routing chunks. Logical offsets survive this slicing. When the complete history fits $L_{\max}$, one core with historical positions recovers the exact dense path. Beyond it, bounded local positions and finite overlap are explicitly an approximation; pretrained integrations must preserve model-native position and cache-layout conventions.

A.5 J. Long Prompts and Streaming Rollover through `#__head`

The implicit head is not a globally named document. It is an internal URI scoped to one request row. The recent tail remains ordinary local context, while displaced leading IDs are encoded through the same cache path as references.

```
055 | def prepare_or_roll(ids, direct_limit, row_cache):
056 |     head_ids, direct_ids = ids[:-direct_limit], ids[-direct_limit:]
057 |     if head_ids:
058 |         row_cache.replace(HEAD_URI, encode_bounded(HEAD_URI, head_ids,
059 |             config.encoding_chunking, config.routing_chunking, Lmax))
060 |     return head_ids, direct_ids
061 |
062 | def append_generated(head_ids, direct_ids, token):
063 |     direct_ids.append(token)
064 |     if len(direct_ids) > direct_limit:
065 |         head_ids += pop_prefix(direct_ids, route_tokens)
 065a|          rebuild_head(head_ids)
```

Splitting occurs on exact token IDs, not decoded and re-tokenized text. Lines 58–59 reuse the bounded reference path, including overlap and logical offsets. Lines 62–66 migrate complete routing-sized prefixes only after an append overflows the direct limit. The prototype rebuilds the head and exact packed index after invalidation; an incremental append implementation may replace line 65 without changing semantics. A batched wrapper holds one `row_cache` per sample, so the identical internal URI cannot cross rows.

A.6 D–E. Gist Construction and Packed Exact Index

For mean routing, each chunk gist is the mean of its projected token keys after heads are merged. Multi-gist modes replace that one vector with G paired prototypes. The router normalizes and packs all layer-local gist sets once; padding exists only in the routing index and is masked before aggregation.

```
066 | def mean_gist(k, v):                  # k,v: [1,H,M,Dh]
067 |     keys = merge_token_heads(k)       # [M,D]
068 |     vals = merge_token_heads(v)       # [M,D]
069 |     return keys.mean(0)[None, :], vals.mean(0)[None, :]
070 |
071 | def pack_index(chunks):
```

```
072 |     sets = [c.routing_gists.k for c in chunks]  # each [Gi,D]
073 |     counts = tensor([len(g) for g in sets])
074 |     flat = cat(sets, dim=0)                     # [sum(Gi),D]
075 |     owner = repeat_interleave(arange(len(sets)), counts)
076 |     slot = cat([arange(n) for n in counts])
077 |     packed = zeros(len(sets), max(counts), D)   # [C,Gmax,D]
078 |     packed[owner, slot] = flat
079 |     mask = arange(max(counts))[None, :] < counts[:, None]
080 |     return normalize(packed, dim=-1), mask
081 |
082 | def score_all(query, packed, mask, reduction="max"):
083 |     query = normalize(query, dim=-1)            # [B,D]
084 |     score = einsum("bd,cgd->bcg", query, packed)
085 |     score = score.masked_fill(~mask[None, :, :], -inf)
086 |     winner_score, winner_gist = score.max(-1)   # both [B,C]
087 |     chunk_score = reduce_gists(score, mask, reduction)
088 |     return chunk_score, winner_gist, winner_score
```

The single-gist implementation specializes line 84 to `query @ packed[:,0,:].T`. Crucially, no candidate score is converted to a Python scalar inside lines 82–88. Detached caches retain this packed index across requests; cache insertion, invalidation, or gist rebuilding must invalidate it. Normal routing transfers only selected identifiers and scores for record assembly, while a diagnostic mode may request the complete ranking. Trainable-gist mode must either rebuild per forward or preserve the autograd graph.

A.7 F. Hierarchical Exact `torch.topk` Routing

Chunk scores are first reduced within each URI, then two independent budgets are applied: k_r references and k_c chunks per selected reference. Confusing these budgets changes both semantics and cost.

```
089 | def hierarchical_route(chunk_score, chunks_by_ref, kr, kc):
090 |     # chunks_by_ref: padded [R,Cr], values are global chunk rows
091 |     grouped = chunk_score[:, chunks_by_ref.clamp_min(0)]  # [B,R,Cr]
092 |     valid = chunks_by_ref[None, :, :] >= 0
093 |     grouped = grouped.masked_fill(~valid, -inf)
094 |     ref_score = grouped.max(-1).values        # or mean/logsumexp
095 |     _, ref_idx = torch.topk(ref_score, min(kr, R), dim=-1)
096 |     chosen = gather_ref_rows(grouped, ref_idx)  # [B,kr,Cr]
097 |     _, local_idx = torch.topk(chosen, min(kc, Cr), dim=-1)
098 |     selected = gather_selected_chunks(ref_idx, local_idx)
099 |     return selected                           # list per batch row
```

Lines 95–97 are exact, not approximate-nearest-neighbor search. Line 96 gathers only the selected reference rows before chunk top-k; this is important because scoring every chunk remains exact while host serialization and second-stage work scale with k_r. These operations remove per-candidate CUDA synchronization while preserving cosine scores and policy. Stable URI and chunk identifiers should resolve exact score ties. `reference_first` routing uses URI-level gists before detailed chunk scoring; global-chunk routing ranks chunks first while enforcing distinct-URI limits. These are alternative policies, not synonyms for the hierarchy above.

A.8 G–H. Budgeted Selection and Full-Chunk Materialization

Routing returns lightweight records. Default materialization then gathers complete native token K/V for the selected chunks. Thus G routing gists do not imply G attended memory positions.

```
100 | def materialize(candidates, direct_tokens, Lmax, reserve, cap):
101 |     budget = min(cap, Lmax - direct_tokens - reserve)
102 |     assert budget >= 0
103 |     ranked = sort(candidates, key=(-score, uri, chunk_id))
104 |     selected, remaining = [], budget
105 |     for hit in deduplicate_and_expand_mode(ranked):
106 |         cost = materialized_token_count(hit)  # whole chunk by default
107 |         if cost > remaining: continue         # later small chunks may fit
108 |         selected.append(hit); remaining -= cost
109 |     assert sum(map(materialized_token_count, selected)) <= budget
110 |     # Only now gather/move [1,H,Mi,Dh] native K/V for selected chunks.
111 |     return gather_native_kv(selected), budget - remaining
```

The candidate list in line 100 already combines implicit history and every explicit source for one row; no source obtains a separate budget. Score ordering occurs before device movement, so line 107 also prevents rejected host K/V from being transferred. The default path therefore has selected length $M_{\text{sel}} = \sum_{c \in A} |c|$ after overlap deduplication. Optional `gist_only` materializes only the winning paired gist K/V; optional `full_reference` expands a selected URI to all its chunks. Those alternatives must be reported separately because they alter the attended information, not merely an implementation detail. With CPU residency, line 110 leaves unselected payload on the host. Transfer bytes count actual device moves, not logical K/V size; GPU-resident selected K/V therefore reports zero transfer.

A.9 I. One-Softmax Native Attention

Each row may select a different memory length M_i. The simplest correct implementation loops over rows; a production kernel may bucket or fuse rows, but it must preserve row ownership and masks.

```
112 | def native_attention(q, local_k, local_v, memory_by_row, valid_mask):
113 |     # q,local_k,local_v: [B,H,T,Dh]
114 |     rows = []
115 |     causal = tril(ones(T, T, dtype=bool))
116 |     for b, (mem_k, mem_v) in enumerate(memory_by_row):
117 |         # mem_k,mem_v: [1,H,Mb,Dh], already historical
118 |         all_k = cat([mem_k, local_k[b:b+1]], dim=2)
119 |         all_v = cat([mem_v, local_v[b:b+1]], dim=2)
120 |         score = q[b:b+1] @ all_k.transpose(-2, -1) / sqrt(Dh)
121 |         memory_visible = ones(T, mem_k.shape[2], dtype=bool)
122 |         visible = cat([memory_visible, causal], dim=1)
123 |         visible[:, mem_k.shape[2]:] &= valid_mask[b][None, :]
124 |         weight = softmax(score.masked_fill(~visible, -inf), dim=-1)
125 |         rows.append(weight @ all_v)
126 |     return cat(rows, dim=0)                    # [B,H,T,Dh]
```

Memory precedes local K/V in lines 118–119 and is visible to every tail query in line 121. The local suffix remains causal in line 122. There is exactly one softmax in line 124, so memory competes directly with local context. The enclosing attention module applies the original output projection after merging heads. If every row has empty memory, it returns the ordinary local-attention result exactly.

A.10 Batch Isolation and End-to-End Assembly

The orchestration layer prepares one namespace per row, wraps those namespaces without flattening duplicate URI strings, and routes `query[b]` only against `row_cache[b]`.

```
127 | row_caches, direct_rows, offsets = [], [], []
128 | for sample in minibatch:
129 |     cache = SimpleCache()
130 |     direct, _head, offset = prepare_long_prompt(sample.ids, limit, cache)
131 |     for handle in sample.references:
132 |         ensure_cached(handle.uri, cache, shared_budget(), [])
133 |     row_caches.append(cache)
134 |     direct_rows.append(direct)
135 |     offsets.append(offset)
136 |
137 | batch_cache = BatchCache(row_caches)
138 | ids, mask = right_pad(direct_rows)
139 | model.set_cache(batch_cache)
140 | logits = model(ids, attention_mask=mask,
141 |                position_offset=tensor(offsets, device=ids.device))
142 | loss = cross_entropy(logits, labels, ignore_index=PAD)
```

Line 137 must not merge entries by URI: two samples may use the same URI string for different authorized content, and every implicit head uses the same internal URI. The preparation loop does not require one model forward per sample; direct tails remain a rectangular batch. Line 141 preserves each row's historical position independently. A vectorized cache builder is valid only if it preserves this ownership.

A.11 Diagnostics and Replication Checks

A replication should test the mechanism in layers rather than relying on final loss alone. First, converted SelfAttention and PRA must match exactly with memory disabled. Second, historical native-all must match dense full-context tail logits. Third, oracle selection tests transport independently of routing. Fourth, valid, shuffled, irrelevant, and disabled conditions test content dependence. Finally, routing reports candidate count, MRR, recall/coverage at k, selected chunks, physical and unique K/V tokens, overlap, actual transfer bytes, index-build and warm-search latency, cache-resident CPU/GPU bytes, peak CUDA allocation/reservation, and separately synchronized routing, materialization, transfer, and memory-attention time.

The configuration fields belong to different moving parts: encoding strategy and position mode determine K/V semantics; chunk size and overlap determine addressable units; gist mode and count determine routing representations; search strategy and k_r/k_c determine selection; materialization mode determines exposed detail; and memory transport determines how selected K/V enters attention. Reporting only "PRA enabled" is therefore insufficient for replication.

References

[1] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2025.

[2] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[3] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George B. M. van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, 2022.

[4] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, 2019.

[5] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A. Smith, and Matt Gardner. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of NAACL*, 2021. Preprint: `https://arxiv.org/abs/2105.03011`.

[6] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Retrieval augmented language model pre-training. In *International Conference on Machine Learning*, 2020.

[7] Dongseong Hwang, Weiran Wang, Zhuoyuan Huo, Khe Chai Sim, and Pedro Moreno Mengibar. TransformerFAM: Feedback attention is working memory. *arXiv preprint arXiv:2404.09173*, 2024.

[8] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *International Conference on Learning Representations*, 2020.

[9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*, 2023. Preprint: `https://arxiv.org/abs/2309.06180`.

[10] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.

[11] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, 2024. Preprint: `https://arxiv.org/abs/2404.14469`.

[12] Guangda Liu, Chengwei Li, Jieru Zhao, Chenqi Zhang, and Minyi Guo. ClusterKV: Manipulating LLM KV cache in semantic space for recallable compression. *arXiv preprint arXiv:2412.03213*, 2024.

[13] Amirkeivan Mohtashami and Martin Jaggi. Landmark attention: Random-access infinite context length for transformers. *arXiv preprint arXiv:2305.16300*, 2023.

[14] Tsendsuren Munkhdalai, Manaal Faruqui, and Siddharth Gopal. Leave no context behind: Efficient infinite context transformers with infini-attention. *arXiv preprint arXiv:2404.07143*, 2024.

[15] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.

[16] Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with routing transformers. *Transactions of the Association for Computational Linguistics*, 9:53–68, 2021. Preprint: `https://arxiv.org/abs/2003.05997`.

[17] Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022.

[18] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. StreamingLLM: Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

[19] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018. Preprint: `https://arxiv.org/abs/1809.09600`.

[20] Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.

[21] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023. OpenReview version: `https://openreview.net/forum?id=RkRrPp7GKO`.